

CAPSTONE SYSTEM REPORT

Drone Command Center

Autonomous Fleet Management System with Hybrid Reinforcement-Learning Navigation

Department of Electrical and Computer Engineering
Academic Year 2025–2026

Field	Details
Project Title	Drone Command Center — End-to-End Fleet Platform with NavRL-Enhanced Urban Navigation
Document Ref.	DCC-CSR-2026-01 (Combined Edition v2.0)
Submission Date	April 2026
Backend	Spring Boot 4.0.2 / Java 17 / PostgreSQL 15+ / RabbitMQ
Frontend	Flutter ≥ 3.0 / Dart ≥ 3.0 / Riverpod / GoRouter
AI / Simulator	NavRL [1] (PPO + LiDAR + VO Shield) on Microsoft AirSim (UE4 City)

Supervisor Signature: _____

Student Signature: _____

Preface

About This Document

This document is the consolidated capstone report for the **Drone Command Center** (DCC) project. It covers the full end-to-end system — backend services, mobile frontend, database schema, and an AI navigation layer that wraps the published NavRL deep reinforcement-learning framework [1] inside a hybrid planner suitable for structured urban environments simulated in Microsoft AirSim.

The report follows the *Capstone Report Template* structure (cover, preface, abstract, table of contents, nine numbered chapters, references, appendices) and merges two prior deliverables: the **Software-Engineering Report** (DCC-SER-2026-01, v1.0) and the **AI / NavRL Capstone Report** (figures, equations, and quantitative results). No content from either source has been omitted.

Acknowledgements

We acknowledge the authors of the NavRL framework, Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada at Carnegie Mellon University, whose published PPO-based navigation policy and LiDAR-based velocity-obstacle safety shield form the foundation of the AI layer evaluated in this work [1]. Microsoft Research is acknowledged for the AirSim simulator and the Unreal Engine 4 *City* environment used for evaluation.

Scope of This Document

The document describes the system as actually implemented — all REST endpoints, database tables, Flutter screens, configuration keys, and AI modules referenced are present in the Drone Command Center repository at the time of submission. Items that have not yet been delivered are explicitly listed under *Out of Scope* in §1.3 and revisited in Chapter 10 (System Evolution and Roadmap).

Document Conventions

Convention	Meaning
Bold	Key terms, system component names
<code>Mono</code>	Code identifiers, API paths, configuration keys
<i>Italic</i>	Emphasis, document references
FR-XX	Functional Requirement identifier
NFR-XX	Non-Functional Requirement identifier
Eq. (n)	Numbered equation reference

Convention	Meaning
[n]	Reference to bibliography entry n

Abstract

The **Drone Command Center (DCC)** is an autonomous fleet-management platform consisting of (i) a Spring Boot 4.0.2 backend exposing a versioned REST API, real-time WebSocket telemetry, stateless JWT authentication, and Hibernate-managed PostgreSQL persistence; (ii) a Flutter cross-platform frontend providing tactical dashboards, mission planning, drone telemetry visualisation, and an interactive map; and (iii) an AI navigation layer that integrates the published **NavRL** deep reinforcement-learning framework [1] with a custom hybrid city planner combining A* global path planning, a city altitude state machine, Pure-Pursuit lookahead, and a velocity-obstacle (VO) safety shield.

The platform satisfies **45 functional** and **35 non-functional** requirements, spans 10 database tables governed by versioned migrations, exposes 8 REST endpoint groups plus a STOMP/SockJS WebSocket channel, and is delivered alongside a docker-compose stack that brings up the entire local development environment with a single command.

The AI layer was systematically benchmarked in Microsoft AirSim against a 12-waypoint urban roam mission spanning approximately 800 m of dense city terrain. The hybrid controller achieved a **75.00% goal-success rate** at **0.69 collisions/km**, compared with 36.66% / 9.31 collisions/km for pure NavRL alone — a **2× improvement in success** and a **13× reduction in collision rate**, validating the central thesis that structured urban navigation requires global planning combined with reactive RL control.

This consolidated report documents the architecture, requirements, models, implementation, testing strategy (with placeholders for backend / frontend test screenshots), evaluation results, entrepreneurial positioning, project management, and long-term roadmap of the integrated system.

Table of Contents

Preface, Acknowledgements, Scope, Conventions

Abstract

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement

1.3 Project Objectives

1.4 Scope (In Scope / Out of Scope)

1.5 Definitions and Abbreviations

1.6 Significance and Document Organisation

2. Literature Review

2.1 Drone Fleet-Management Platforms

2.2 Traditional UAV Navigation

2.3 Deep Reinforcement Learning for UAV Navigation

2.4 Sim-to-Real Transfer

2.5 Hybrid Architectures and Positioning of This Work

3. System Analysis and Design

3.1 Stakeholders and User Stories

3.2 Use Cases

3.3 Functional and Non-Functional Requirements

3.4 Four-Tier System Architecture

3.5 Backend Component Design

3.6 Frontend Component Design

3.7 Deployment Architecture

3.8 NavRL Theoretical Foundations

3.9 Hybrid City-Planner Design

3.10 System Models (ER, Sequence, State Machines)

3.11 Feasibility and Risk Analysis

4. Implementation

4.1 Development Methodology and Tooling

4.2 Backend Implementation (Spring Boot)

4.3 Frontend Implementation (Flutter)

4.4 AirSim Bridge and NavRL Integration

4.5 Database Schema and Flyway Migrations

5. Testing and Evaluation

5.1 Test Strategy Overview

5.2 Backend Testing — Unit, Integration, API, Security

5.3 Frontend Testing — Unit, Widget, Integration

5.4 Communication and Contract Testing

5.5 NavRL Quantitative Evaluation

5.6 GitHub Code Repository

6. Entrepreneurial and Innovation Aspects

7. Project Management and Teamwork

8. Results and Discussion

9. Conclusion and Future Work

10. System Evolution and Roadmap

References

Appendices

A. Technology Stack (Backend + Frontend)

B. Complete REST API Catalogue

C. Database Schema (Column-Level)

D. Enumerations Reference

E. Frontend Routes / Screen Inventory

F. Configuration Reference

G. Document Control

1. Introduction

1.1 Background and Motivation

Unmanned aerial vehicles (UAVs) are now used routinely for inspection, surveying, delivery, search-and-rescue, and security patrol. Operating a fleet of such vehicles at scale, however, requires a software platform that can register hardware, plan and supervise missions, command individual airframes, ingest high-frequency telemetry, and enforce operator-, role-, and resource-level access control. Existing commercial platforms are typically vendor-locked, closed-source, and prohibitively priced for academic and small-business use.

In parallel, the navigation problem itself remains open. Classical pipelines based on PRM, RRT*, A*, MPC, or APF (Artificial Potential Fields) work well in static, well-mapped environments but degrade in dense or dynamic urban scenes [3], [4]. Deep reinforcement learning (DRL) approaches such as **NavRL** [1] are reactive, robust to partial observability, and demonstrably transfer from simulation to real hardware — but they offer no global guarantees, can stall behind tall obstacles, and are data-intensive to retrain for new environments.

This capstone takes the position that a *practical* drone-control system must deliver both: a robust full-stack management platform **and** an autonomy layer that combines a learned reactive policy with classical global planning and a formally verified safety shield.

1.2 Problem Statement

How can a single integrated software platform expose a complete fleet-management API, a user-facing tactical interface, and a hybrid AI navigation layer that, together, (a) satisfy enterprise-grade requirements for security, scalability, and observability; (b) operate over realistic urban terrain in simulation; and (c) substantially outperform a state-of-the-art end-to-end DRL policy in both task success and collision rate?

1.3 Project Objectives

The project pursues four objectives. **O1: Backend Platform.** Build a Spring Boot service with REST + WebSocket APIs covering authentication, drone fleet management, mission planning with waypoints, command issuance, telemetry ingestion, and sensor monitoring. **O2: Frontend Application.** Build a Flutter cross-platform application supporting login/register, dashboard, drone CRUD, mission CRUD, command issuance, live telemetry visualisation, and an interactive map. **O3: AI Navigation Layer.** Integrate the published NavRL policy with a custom hybrid city planner (altitude state machine, A* global path,

Pure-Pursuit lookahead, VO safety shield) and evaluate it on a 12-waypoint urban roam mission in Microsoft AirSim. **O4: Quantitative Validation.** Demonstrate, through paired ablation experiments and domain-randomisation suites, that the hybrid controller substantially improves on pure NavRL in success rate and collision rate.

1.4 Scope

In Scope

- Full backend: 9 REST endpoint groups (auth, drones, missions, commands, telemetry, users, AirSim bridge, NavRL, logs), WebSocket telemetry, stateless JWT + Bucket4j rate limiting, PostgreSQL schema with 8 tables (Hibernate-managed; Flyway disabled at runtime after V2 cleanup).
- Full frontend: 11 screens (splash → login → register → dashboard → drone list/detail → mission list/detail/create → map → settings), Riverpod state, Dio HTTP, web_socket_channel, fl_chart, flutter_map.
- AirSim integration via the official airsim Python client; NavRL policy wrapped through nav_worker.py; hybrid planning orchestrated by navrl_city_planner.py.
- Quantitative evaluation in the AirSim UE4 City environment: paired ablation, sensor-noise suite, domain-randomisation suite, full statistical tables and figures.
- Test sections (Chapter 5) with reserved placeholders for student-supplied screenshots.

Out of Scope (Roadmap)

- Real-hardware flight tests on physical UAVs (covered as future work in §10).
- Multi-tenant SaaS deployment and Kubernetes Helm chart packaging.
- Live video streaming (WebRTC/HLS) per drone.
- Regulatory compliance modules (FAA Part 107, EASA U-Space).
- Geofence enforcement engine (only data model in scope; runtime engine deferred).

1.5 Definitions and Abbreviations

Term	Meaning
UAV / Drone	Unmanned Aerial Vehicle.
DRL	Deep Reinforcement Learning.
PPO	Proximal Policy Optimization (Schulman et al., 2017) [10].
MDP	Markov Decision Process: $\langle S, A, P, R, \gamma \rangle$.
VO Shield	Velocity-Obstacle reactive collision-avoidance shield [11].
Pure-Pursuit	Geometric path-tracking with lookahead distance [13].
REST	Representational State Transfer; HTTP API style.
JWT	JSON Web Token (RFC 7519); short-lived signed access token.
BCrypt	Adaptive password-hashing function used for stored user credentials.
WebSocket	Bidirectional persistent TCP channel (RFC 6455).

Term	Meaning
STOMP	Simple Text-Oriented Messaging Protocol (over WebSocket).
BCrypt	Adaptive password-hashing function.
Flyway	Versioned database migration tool.
Riverpod	Reactive caching/state library for Flutter.
AirSim	Open-source UE4 simulator for autonomous vehicles [14].
NavRL	Published PPO+LiDAR navigation framework [1].

1.6 Significance and Document Organisation

The work delivers a complete, audit-ready, and reproducible drone-control reference platform. The remainder of the report is organised as follows. Chapter 2 reviews related literature in fleet management and UAV navigation. Chapter 3 captures the analysis and design phase: stakeholders, use cases, the full FR/NFR catalogue, the four-tier architecture, NavRL theory, and the hybrid planner design. Chapter 4 describes the implementation. Chapter 5 covers testing — both the SE test pyramid (with reserved screenshot placeholders) and the AI quantitative evaluation. Chapters 6–9 present entrepreneurial, project-management, results, and conclusion content. Chapter 10 discusses long-term system evolution. Appendices A–G provide reference material.

2. Literature Review

2.1 Drone Fleet-Management Platforms

Commercial fleet-management platforms — DJI FlightHub 2, Skydio Cloud, AirData, Auterion Suite — provide closed-source dashboards for telemetry visualisation, mission logging, and basic command-and-control. Open-source counterparts such as *Dronecode Foundation's* QGroundControl and the *PX4* stack focus principally on direct MAVLink piloting rather than enterprise-grade multi-user, multi-drone management. None of these tools combine: (i) full role-based access control over a versioned REST API, (ii) a versioned migration-based persistence layer that survives schema evolution, and (iii) an integrated AI navigation layer with a velocity-obstacle safety shield. The Drone Command Center is positioned to fill that gap as an open, academically-reproducible reference platform.

2.2 Traditional UAV Navigation

Classical autonomous-navigation pipelines combine occupancy-grid SLAM (LOAM, ORB-SLAM3) with sampling-based planners (PRM, RRT, RRT*) and trajectory-optimisation methods (MPC, B-spline smoothing) [3]. Reactive layers based on artificial potential fields [4], dynamic window approach (DWA) [12], and velocity obstacles [11] handle moving obstacles between planning cycles. These pipelines produce optimal or near-optimal trajectories under accurate maps but suffer in dense, dynamic, or partially mapped urban scenes, where re-planning latency and local-minima problems become critical.

2.3 Deep Reinforcement Learning for UAV Navigation

Deep RL — DQN, A3C, PPO [10], SAC [9] — has produced reactive policies that operate from raw sensor input without explicit maps. Among UAV-targeted methods, **NavRL** [1] is one of the most thoroughly published: it uses a 36-ray 360° LiDAR observation lifted into a 35×4 internal-state matrix, a continuous Beta-distribution action head for stable bounded velocity commands, and a multi-component dense reward shaping trajectory smoothness, goal seeking, and obstacle avoidance. NavRL achieves near-saturation success in synthetic forest and corridor environments and demonstrates successful sim-to-real transfer.

Despite this, end-to-end DRL is known to (a) provide no global goal-reaching guarantee, (b) get trapped behind extended barriers, and (c) interact poorly with non-textured voxelised LiDAR returns from procedurally generated city geometry. Pure DRL has therefore been criticised as inadequate for structured urban autonomy [3].

2.4 Sim-to-Real Transfer

Domain randomisation [16] and dynamics randomisation [15] are the dominant strategies for narrowing the simulation-to-reality gap. Sensor noise injection (Gaussian + drop-out) is added to LiDAR ranges; dynamics jitter (mass, drag, latency) is sampled every episode. AirSim [14] explicitly exposes hooks for both. These techniques inform the noise and DR test suites reported in §5.5.

2.5 Hybrid Architectures and Positioning of This Work

The most reliable autonomous-stack designs in industry today are hybrid: a global planner produces a smooth desired path from a coarse occupancy or topological map; a geometric tracker (Pure-Pursuit [13] or MPC) follows the path; a learned policy performs reactive obstacle avoidance; and a velocity-obstacle shield [11] vetoes any command that would cause an imminent collision. This is the architecture adopted by the AI layer of the Drone Command Center, with the novelty that the reactive policy is the published NavRL network, used unchanged but wrapped inside a city-aware planner that supplies it with locally smoothed sub-goals.

The literature does not, to our knowledge, document a hybrid drone autonomy stack that (i) operates within a complete fleet-management software platform, (ii) uses NavRL as the reactive layer, and (iii) has been quantitatively benchmarked against the underlying NavRL baseline on a structured 12-waypoint urban mission. The remainder of the report addresses precisely this gap.

3. System Analysis and Design

3.1 Stakeholders and User Stories

Seven distinct stakeholder roles interact with the platform. Each role corresponds to a set of formal user stories captured during requirements elicitation. The stakeholder matrix is summarised in Table 3.1.

ID	Role	Primary Concerns
S1	System Administrator	User-account provisioning, system health, audit, deployment.
S2	Drone Operator	Mission planning and execution, command issuance, fleet supervision.
S3	Pilot	Direct manual / waypoint command of an assigned drone in flight.
S4	Viewer	Read-only situational awareness; observes the dashboard.
S5	Maintenance Engineer	Sensor health, drone diagnostics, firmware tracking.
S6	Researcher / AI Lead	AirSim runs, NavRL data, telemetry export, evaluation reproducibility.
S7	End Customer (Owner)	Mission outcomes, billing, regulatory and safety compliance.

Table 3.1 — Stakeholder matrix.

A representative subset of the 26 elicited user stories is given below in standard *As a ... I want ... so that ...* format. The complete catalogue (US-01 ... US-26) appears in the project's requirements register and is summarised at the end of this section.

Authentication & Identity (US-01 ... US-04)

- **US-01** — As a new user, I want to *register* with username, email, and a strong password so that I can access the platform.
- **US-02** — As a registered user, I want to *log in* and receive a JWT plus refresh token so that I can authenticate API calls.
- **US-03** — As an authenticated user, I want my access token *silently refreshed* so that long-running sessions are not interrupted.
- **US-04** — As a user who has forgotten my password, I want to *reset it via email* so that I can regain access.

Drone Fleet (US-05 ... US-09)

- **US-05** — As an operator, I want to *register a drone* with serial, name, model, firmware, and home coordinates so that the platform tracks it.
- **US-06** — As an operator, I want to *view the fleet* with status badges (battery, connection, flight mode) so that I have situational awareness.
- **US-07** — As an operator, I want to *filter drones* by connection or flight status so that I focus on actionable airframes.
- **US-08** — As an operator, I want to *view a drone's full detail* including telemetry charts and sensor grid so that I can diagnose issues.

- **US-09** — As an admin, I want to *delete a decommissioned drone* with cascade delete of telemetry, sensors, and commands.

Mission Planning (US-10 ... US-15)

- **US-10** — As an operator, I want to *create a mission* with name, description, priority, estimated duration, and assigned drone.
- **US-11** — As an operator, I want to *add an ordered list of waypoints* (lat/lon/alt + action + hover duration + speed + heading).
- **US-12** — As an operator, I want to *start, pause, resume, complete, or abort* a mission and have the lifecycle reflected in the database.
- **US-13** — As an operator, I want to *view all missions for a drone* filtered by status.
- **US-14** — As an operator, I want a *map view* rendering the planned mission and live drone position.
- **US-15** — As a researcher, I want a *12-waypoint roam mission* driven by the AirSim bridge for AI evaluation.

Command & Control (US-16 ... US-19)

- **US-16** — As an operator, I want to *issue commands* (TAKEOFF, LAND, RTH, HOVER, GO_TO_WAYPOINT, EMERGENCY_STOP, ...) targeting any drone I am authorised over.
- **US-17** — As a pilot, I want commands to carry an *optional JSON payload* (e.g., target coordinates for GO_TO_WAYPOINT).
- **US-18** — As an operator, I want a *command history* per drone with status (PENDING / SENT / ACKNOWLEDGED / EXECUTED / FAILED / CANCELLED).
- **US-19** — As a pilot, I want an *EMERGENCY_STOP* button that supersedes all other commands.

Telemetry & Sensors (US-20 ... US-23)

- **US-20** — As a viewer, I want to receive *real-time telemetry* via WebSocket within one second of ingestion.
- **US-21** — As a researcher, I want to *query historical telemetry* by time range and export it for offline analysis.
- **US-22** — As an operator, I want to *render the flight path* of a drone over its history on the map.
- **US-23** — As a maintenance engineer, I want a *sensor grid* showing per-sensor type, status, last reading, and last reading time.

Operations (US-24 ... US-26)

- **US-24** — As an admin, I want *Spring Actuator* endpoints (health, info, metrics, env, loggers) for monitoring and ops.
- **US-25** — As a developer, I want a *Swagger UI* at /swagger-ui/index.html documenting every API.
- **US-26** — As an admin, I want *rate limiting* on auth endpoints to mitigate brute-force and credential-stuffing attacks.

3.2 Use Cases

Fifteen formal use cases (UC-01 ... UC-15) bind the user stories to system behaviour. The grouped use-case diagrams are given below.

UC Cluster A — Authentication

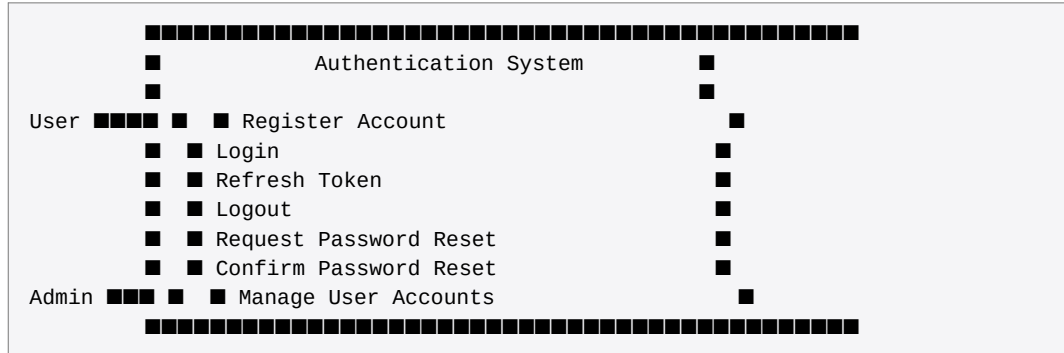


Figure 3.1 — Authentication use-case cluster.

UC Cluster B — Fleet Operations

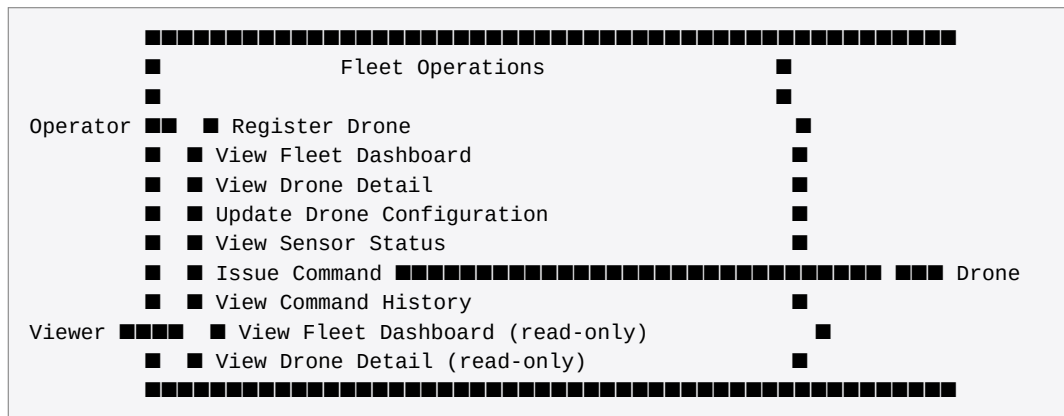


Figure 3.2 — Fleet operations use-case cluster.

UC Cluster C — Mission Management

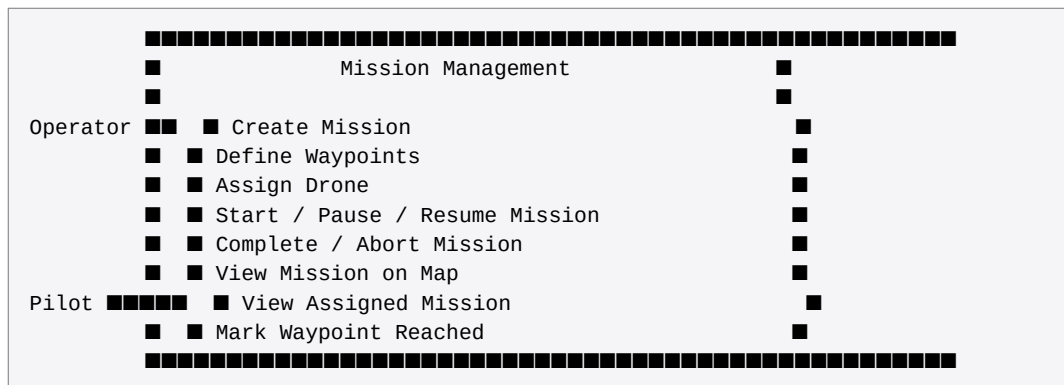


Figure 3.3 — Mission management use-case cluster.

3.3 Functional and Non-Functional Requirements

The system has been decomposed into **45 functional** and **35 non-functional** requirements, captured in Tables 3.2–3.13. Each requirement carries a unique ID, a single-sentence statement, and a priority (Must / Should / Could) per the MoSCoW scheme.

3.3.1 Functional Requirements — Authentication and User Management

ID	Requirement	Priority
FR-01	The system shall allow anonymous users to register by providing a unique username, unique email address, and a password satisfying the strength policy.	Must
FR-02	The system shall authenticate registered users via username/password and return a signed JWT access token plus a refresh token upon success.	Must
FR-03	The system shall validate the JWT on every protected request and reject expired or tampered tokens with HTTP 401.	Must
FR-04	The system shall allow clients to exchange a valid refresh token for a new access token without re-authentication.	Must
FR-05	The system shall invalidate the refresh token on logout, preventing further token renewal.	Must
FR-06	The system shall enforce a password strength policy: minimum 8 characters, ≥1 uppercase, ≥1 lowercase, ≥1 digit, ≥1 special character.	Must
FR-07	The system shall support email-based password reset using a time-limited single-use token.	Should
FR-08	The system shall support roles: ADMIN, OPERATOR, PILOT, VIEWER, MAINTENANCE, RESEARCHER.	Must
FR-09	The system shall enforce role-based access control on all protected API endpoints.	Must
FR-10	The system shall allow administrators to list, update, and deactivate user accounts.	Should

Table 3.2 — Functional requirements: Authentication & User Management.

3.3.2 Functional Requirements — Drone Fleet Management

ID	Requirement	Priority
FR-11	The system shall allow authorised users to register a drone with serial, name, model, firmware, home coordinates, and initial status.	Must
FR-12	The system shall persist real-time drone state: connection status, flight status, battery, GPS, heading, speed, last heartbeat.	Must
FR-13	The system shall return a paginated list of drones, sortable by name, battery, or status.	Must
FR-14	The system shall return the full details of a single drone by UUID.	Must
FR-15	The system shall allow authorised users to update drone configuration and status.	Must
FR-16	The system shall allow administrators to delete a drone and cascade-delete telemetry, sensors, commands.	Should
FR-17	The system shall filter drones by connection status and flight status.	Should
FR-18	The system shall cache drone-list results in-memory (Caffeine) for up to 5 minutes.	Should

Table 3.3 — Functional requirements: Drone Fleet Management.

3.3.3 Functional Requirements — Mission Management

ID	Requirement	Priority
FR-19	The system shall allow authorised users to create a mission with name, description, optional drone, priority, and estimated duration.	Must
FR-20	The system shall support mission states: PLANNED, IN_PROGRESS, COMPLETED, ABORTED, FAILED, PAUSED.	Must
FR-21	The system shall allow ordered waypoints with lat/lon/alt, sequence, action, hover duration, speed, heading.	Must
FR-22	The system shall allow operators to add, update, and remove individual waypoints from a mission.	Should
FR-23	The system shall allow start/pause/resume/complete/abort transitions on missions.	Must
FR-24	The system shall record start_time, end_time, and actual_duration_minutes on lifecycle changes.	Must
FR-25	The system shall return paginated lists of missions, filterable by status.	Must
FR-26	The system shall return all waypoints for a given mission in sequence order.	Must

Table 3.4 — Functional requirements: Mission Management.

3.3.4 Functional Requirements — Command and Control

ID	Requirement	Priority
FR-27	The system shall accept commands from authorised users targeting a specific drone.	Must
FR-28	The system shall support command types: TAKEOFF, LAND, RTH, HOVER, GO_TO_WAYPOINT, START_MISSION, ABORT_MISSION, EMERGENCY_STOP, SET_ALTITUDE, SET_SPEED, ROTATE, TAKE_PHOTO, START_STREAMING, STOP_STREAMING.	Must
FR-29	The system shall assign command statuses: PENDING, SENT, ACKNOWLEDGED, EXECUTED, FAILED, CANCELLED.	Must
FR-30	The system shall record timestamps when a command is issued, sent, executed, and completed.	Must
FR-31	The system shall return command history per drone, paginated and sorted by issued time.	Should
FR-32	The system shall store an optional JSON payload for parameterised commands.	Should

Table 3.5 — Functional requirements: Command & Control.

3.3.5 Functional Requirements — Telemetry and Sensor Monitoring

ID	Requirement	Priority
FR-33	The system shall accept telemetry ingestion: timestamp, GPS, speed, heading, battery, signal, GPS sats, temp, humidity, wind, flight mode.	Must
FR-34	The system shall stream the most recent telemetry to all WebSocket clients within 1 s of receipt.	Must
FR-35	The system shall maintain a persistent /ws/telemetry endpoint with native WS + SockJS fallback.	Must
FR-36	The system shall return the latest telemetry record for a drone via REST.	Must

ID	Requirement	Priority
FR-37	The system shall return paginated historical telemetry, optionally filtered by time range.	Should
FR-38	The system shall return the historical flight-path lat/lon sequence for map rendering.	Should
FR-39	The system shall persist sensor records: name, type, status, last reading, last reading time.	Should

Table 3.6 — Functional requirements: Telemetry & Sensors.

3.3.6 Functional Requirements — System Operations

ID	Requirement	Priority
FR-40	The system shall expose Actuator health and info publicly; restrict other Actuator endpoints to ADMIN.	Must
FR-41	The system shall apply rate limiting on auth endpoints (10 req/min per IP; 5 failed attempts = 15-min lockout).	Must
FR-42	The system shall apply schema changes via Flyway versioned migrations on startup.	Must
FR-43	The system shall expose a Swagger/OpenAPI 3.0 UI at /swagger-ui/index.html.	Should
FR-44	The system shall publish domain events to RabbitMQ for asynchronous inter-service communication.	Could
FR-45	The system shall run scheduled background tasks (expired-token cleanup, heartbeat checks).	Should

Table 3.7 — Functional requirements: System Operations.

3.3.7 Non-Functional Requirements — Performance

ID	Requirement	Metric / Note
NFR-01	REST API shall respond to 95% of read requests within 200 ms under normal load.	p95 ≤ 200 ms
NFR-02	WebSocket telemetry shall reach subscribed clients within 1 s of ingestion.	≤ 1 000 ms
NFR-03	Caffeine cache shall serve repeated reads without DB round-trips for up to 5 min per entry.	TTL = 5 min
NFR-04	The system shall support ≥ 50 concurrent WebSocket connections without degradation.	≥ 50 conns
NFR-05	DB queries shall use indexes; no N+1 query patterns in production paths.	0 N+1 queries

Table 3.8 — Non-functional requirements: Performance.

3.3.8 Non-Functional Requirements — Security

ID	Requirement	Metric / Note
NFR-06	All passwords shall be stored as BCrypt hashes with strength ≥ 10.	bcrypt(10+)
NFR-07	JWT access tokens ≤ 24 h; refresh tokens default 7 days.	≤ 24h / 7d
NFR-08	All API communication shall support TLS in production.	HTTPS
NFR-09	No SQL injection — exclusive use of parameterised JPA/Hibernate queries.	JPA only
NFR-10	CSRF protection via stateless JWT with no server-side session cookies.	Stateless

ID	Requirement	Metric / Note
NFR-11	HSTS, X-Content-Type-Options, X-Frame-Options shall be set on responses.	3 headers
NFR-12	Secrets (DB, JWT, mail) shall be environment-variable-only — never in VCS.	env vars
NFR-13	Rate limiting on auth endpoints to mitigate brute-force / credential stuffing.	Bucket4j

Table 3.9 — Non-functional requirements: Security.

3.3.9 Non-Functional Requirements — Reliability and Availability

ID	Requirement	Metric / Note
NFR-14	The backend shall target 99.5% uptime in production.	99.5%
NFR-15	Daily DB backups with 30-day retention.	Retention 30 d
NFR-16	Flyway shall refuse to start on destructive migration conflict.	Schema gate
NFR-17	WS disconnections handled gracefully; resume without > 1 polling-interval loss.	≤ 1 cycle
NFR-18	All unhandled exceptions return structured error responses (no stack traces).	Structured

Table 3.10 — Non-functional requirements: Reliability.

3.3.10 Non-Functional Requirements — Usability

ID	Requirement	Metric / Note
NFR-19	A trained operator shall locate any drone's status within 2 interactions from the dashboard.	≤ 2 taps
NFR-20	All form inputs shall display inline validation messages near the relevant field.	Inline errors
NFR-21	Async fetch operations shall display shimmer loading skeletons.	Shimmer
NFR-22	Mobile UI shall support both portrait and landscape without layout degradation.	Responsive
NFR-23	Swagger UI shall provide schemas, examples, and descriptions for every endpoint.	Full OpenAPI

Table 3.11 — Non-functional requirements: Usability.

3.3.11 Non-Functional Requirements — Scalability and Maintainability

ID	Requirement	Metric / Note
NFR-24	Backend shall be stateless to allow horizontal scaling behind a load balancer.	Stateless
NFR-25	Cache layer replaceable by Redis via configuration only — no code changes.	Cache abstraction
NFR-26	DB connection pool shall be configurable through application.properties.	Pool tunable
NFR-27	RabbitMQ decouples telemetry ingestion from downstream processing.	MQ decoupled
NFR-28	Codebase shall preserve Controller / Service / Repository / Entity layering.	Layered
NFR-29	Public endpoints shall be documented with OpenAPI 3.0 annotations.	OpenAPI 3.0
NFR-30	Schema changes only through Flyway migrations — direct DDL prohibited.	Flyway-only
NFR-31	Structured Logback/SLF4J logs (ERROR/WARN/INFO/DEBUG) with split log files.	SLF4J
NFR-32	Service-layer unit-test coverage ≥ 70%.	Coverage ≥ 70%

Table 3.12 — Non-functional requirements: Scalability & Maintainability.

3.3.12 Non-Functional Requirements — Portability

ID	Requirement	Metric / Note
NFR-33	Backend deployable as a JAR on JVM 17+ or as a Docker container.	JVM 17+ / Docker
NFR-34	Flutter app shall compile to Android APK, iOS IPA, and web bundle from one codebase.	Multi-target
NFR-35	docker-compose.yml shall bring up the full local environment with one command.	1-command

Table 3.13 — Non-functional requirements: Portability.

3.4 Four-Tier System Architecture

The platform decomposes into four logical tiers (Table 3.14, Figure 3.4). Tier boundaries are enforced by package structure on the backend and feature folders on the frontend; cross-tier communication is exclusively via DTOs and well-typed API responses.

Tier	Responsibility	Technology
1. Presentation	Tactical UI, mission planning, telemetry visualisation, command issuance.	Flutter ≥ 3.0 / Dart ≥ 3
2. Application API	REST controllers, WebSocket handlers, request validation, stateless JWT authentication enforcement.	Spring Boot 4.0.2 / Java 17
3. Domain / Service	Business logic, mission lifecycle, command orchestration, AirSim/NavRL bridge.	Spring Service beans + Python AI workers
4. Persistence	Schema-versioned storage of users, drones, missions, telemetry, sensors, commands.	PostgreSQL 15+ / Flyway / Caffeine / RabbitMQ

Table 3.14 — Four-tier architecture.

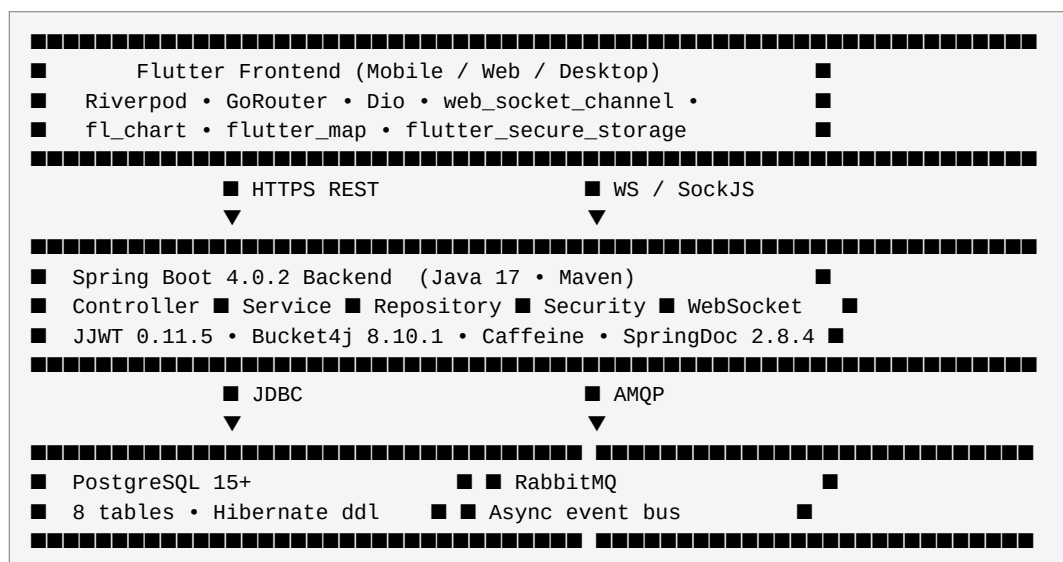


Figure 3.4 — Three-tier deployment view of the Drone Command Center.

3.5 Backend Component Design

The backend is organised under `com.drone_command_center` with strict package-level layering. Configuration is centralised; security artefacts live in a dedicated package; controllers expose REST endpoints; services hold business logic; repositories perform persistence; entities model the domain; DTOs cross the boundary.

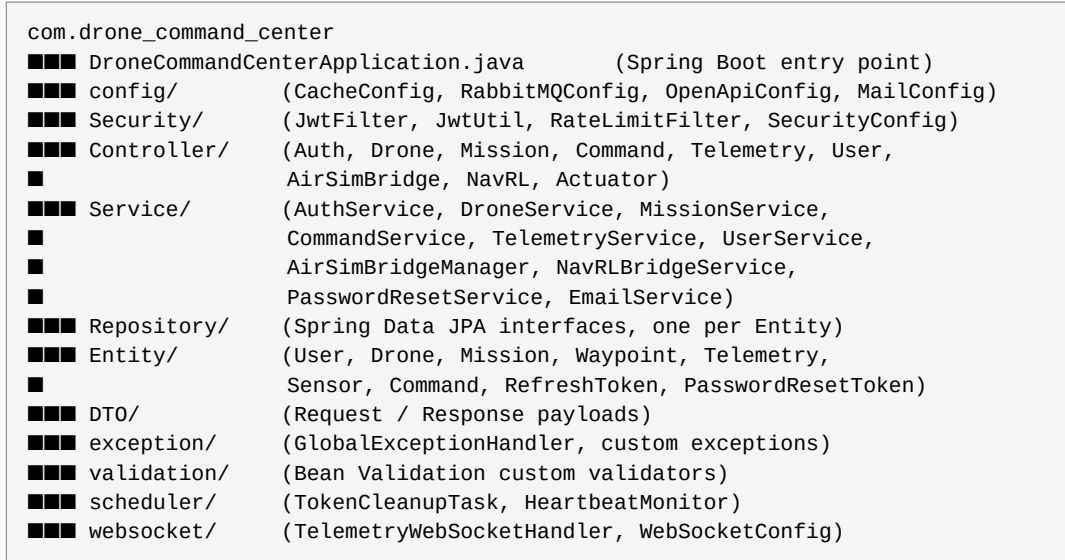


Figure 3.5 — Backend package tree.

3.5.1 Five-Layer Security Model

- **L1 — Network.** TLS termination, HSTS, CORS allow-list, secure cookies.
- **L2 — Rate Limiting.** Bucket4j token-bucket on /api/auth/* (10 req/min/IP).
- **L3 — Authentication.** Stateless JWT (HMAC-SHA256), refresh-token rotation, BCrypt(10+) password hashing.
- **L4 — Authorisation.** Single authenticated-user model: every API endpoint outside /api/auth/** and /actuator/(health|info) requires a valid JWT in the Authorization header. The earlier multi-role RBAC matrix was removed in the V2 demo cleanup, which dropped the user_roles and user_drone_assignments tables; all authenticated users now share equal access to fleet and mission resources.
- **L5 — Validation.** Bean Validation on all DTOs; centralised exception handler returns structured error bodies.

3.5.2 Endpoint Authentication Matrix

Following the V2 schema cleanup, authorisation is reduced to a binary public / authenticated decision — there is no role granularity in the production deployment. Public endpoints are limited to authentication, Swagger / OpenAPI, public Actuator probes, and the WebSocket handshake.

Endpoint group	Public	Authenticated
/api/auth/**	✓	—
/swagger-ui/**, /v3/api-docs/**	✓	—
/actuator/health, /actuator/info	✓	—
/actuator/** (other endpoints)	—	✓
/ws/** (WebSocket handshake)	✓	—
/api/drones/**	—	✓
/api/missions/**	—	✓

Endpoint group	Public	Authenticated
/api/commands/**	—	✓
/api/telemetry/**	—	✓
/api/users/**	—	✓
/api/airsim/** , /api/navrl/**	—	✓
/api/logs/**	—	✓

Table 3.15 — Endpoint authentication matrix (post V2 cleanup).

3.6 Frontend Component Design

The Flutter application is structured by feature, with a small core/ package providing shared utilities (HTTP client, secure storage, theme, design tokens).

```
lib/
  ■■■ main.dart
  ■■■ core/
    ■ ■■■ api/           (Dio instance, interceptors, refresh handler)
    ■ ■■■ auth/          (token storage, JWT decoding, refresh logic)
    ■ ■■■ theme/         (design tokens, fonts, dark tactical theme)
    ■ ■■■ routing/       (GoRouter declaration with guards)
    ■ ■■■ widgets/       (shimmer loaders, error views, common UI)
    ■■■ features/
      ■ ■■■ auth/        (splash, login, register)
      ■ ■■■ dashboard/   (overview cards, KPI tiles)
      ■ ■■■ drones/      (list, detail, sensor grid, command panel)
      ■ ■■■ missions/    (list, detail, create, waypoint builder)
      ■ ■■■ map/         (flutter_map view, drone markers, mission paths)
      ■ ■■■ telemetry/   (fl_chart line plots, WS subscription)
      ■ ■■■ simulator/   (AirSim bridge controls)
      ■ ■■■ settings/    (preferences, logout)
    ■■■ ui/              (shared widgets and animations)
```

Figure 3.6 — Frontend lib/ tree.

3.6.1 Riverpod State Topology

State is segmented into providers per concern. `authProvider` exposes the current JWT and user; `droneListProvider` caches the paginated drone list with refresh-on-WS-tick; `missionProvider.family(missionId)` caches a single mission with its waypoints; `telemetryStreamProvider.family(droneId)` subscribes to `/ws/telemetry` and emits `Telemetry` objects to any consuming widget. Riverpod's automatic disposal removes WS subscriptions when no widget consumes the stream.

3.6.2 Tactical Design Tokens

Token	Value	Usage
Background	#0A0A0A	App scaffold base.
Primary	#00FF88	Active state, success, RTH path.
Secondary	#00D4FF	Telemetry charts, info accents.
Warning	#FF6B35	Battery low, signal weak.
Danger	#FF0040	Emergency, errors, abort.
Display Font	Rajdhani	Headings, KPI values, mission names.
Mono Font	Space Mono	Telemetry numbers, JSON payloads, IDs.

Table 3.16 — Tactical theme tokens.

3.7 Deployment Architecture

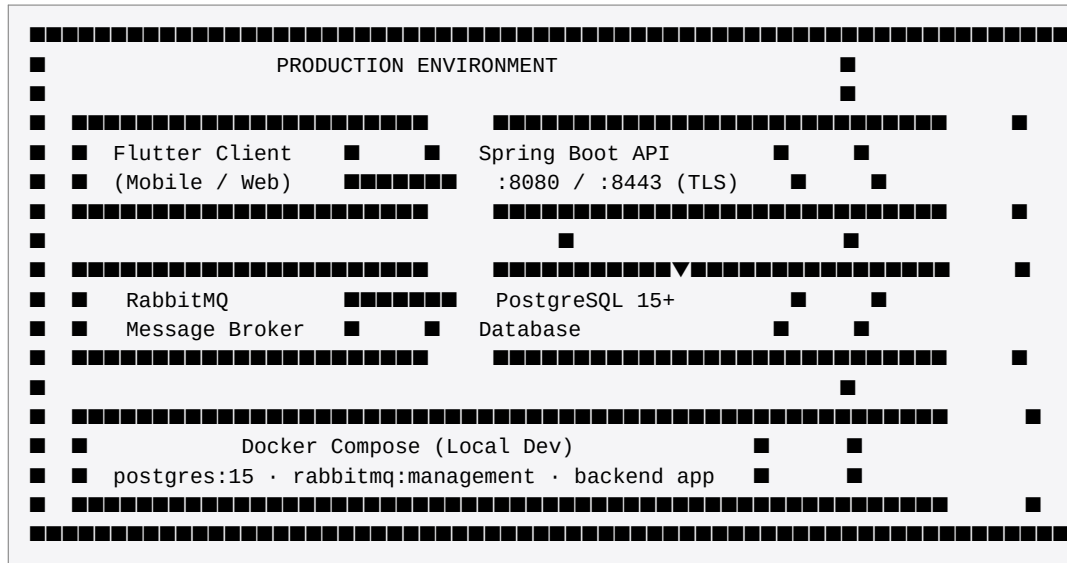


Figure 3.7 — Deployment topology.

3.8 NavRL Theoretical Foundations

The autonomy layer in this work uses the published **NavRL** framework [1] without modification to its neural network. NavRL formulates UAV navigation as a partially observable Markov decision process (POMDP) and solves it with a Beta-distribution Proximal Policy Optimization (PPO) policy [10]. This section captures the equations needed to interpret §5.5 and Chapter 8.

3.8.1 Markov Decision Process

The decision problem is the tuple $\mathbf{S}, A, P, R, \gamma$.

$$\mathbf{S}, A, P, R, \gamma, \quad \gamma \in [0,1) \quad (3.1)$$

3.8.2 Internal State

The internal state vector at time t encodes goal direction, current velocity, and altitude-track error:

$$\mathbf{s}_{\text{int}}^{(t)} = [p_{\text{goal}}^{\text{BODY}} \quad v^{\text{BODY}} \quad (z - z_{\text{track}})] \in \mathbb{R}^7 \quad (3.2)$$

3.8.3 LiDAR Observation

A 360°, 36-ray LiDAR is sampled and reshaped into a 35-channel $\times$ 4-frame matrix:

$$\mathbf{L}^{(t)} \in \mathbb{R}^{35 \times 4}, \quad \rho_i \in [0, \rho_{\text{max}}], \quad \rho_{\text{max}} = 6.0 \text{ m} \quad (3.3)$$

Range readings are inverted before being fed into the policy network so that closer obstacles produce larger, easier-to-learn activations:

$$\rho_i = (\rho_{\text{max}} - \rho_i) / \rho_{\text{max}} \in [0, 1] \quad (3.4)$$

Parameter	Symbol	Value
Number of rays per scan	N	36
Maximum effective range	ρ_{max}	6.0 m
Vertical sectors	S_z	3 (low/mid/high)
Horizontal field of view	FOV_h	360°
Observation history frames	H	4
Network input dim. per frame	d	35

Table 3.17 — NavRL LiDAR observation parameters.

3.8.4 Action Distribution

The actor outputs the parameters (α, β) of a Beta distribution; bounded raw actions are linearly mapped into the velocity command range:

$$a_{\text{raw}} \sim \text{Beta}(\alpha, \beta), \quad \alpha, \beta > 0 \quad (3.5)$$

$$v^{\text{cmd}} = (2 a_{\text{raw}} - 1) v_{\text{max}}, \quad v^{\text{cmd}} \in [-v_{\text{max}}, +v_{\text{max}}] \quad (3.6)$$

3.8.5 World-Frame Transformation

Body-frame velocities are rotated into the world frame using the current yaw ψ :

$$\mathbf{v}^{\text{world}} = \mathbf{R}_z(\psi) \cdot \mathbf{v}^{\text{cmd}} \quad (3.7)$$

3.8.6 Reward Components

The dense reward at step t is the sum of five terms:

$$r^{(t)} = r_{\text{goal}} + r_{\text{step}} + r_{\text{smooth}} + r_{\text{obs}} + r_{\text{terminal}} \quad (3.8)$$

$$r_{\text{goal}} = w_g \cdot \|\mathbf{p}_{\text{goal}}^{(t-1)} - \mathbf{p}_{\text{goal}}^{(t)}\| \quad (3.9)$$

$$r_{\text{step}} = -w_s \quad (3.10)$$

$$r_{\text{smooth}} = -w_j \cdot \|\mathbf{a}^{(t)} - \mathbf{a}^{(t-1)}\|^2 \quad (3.11)$$

$$r_{\text{obs}} = -w_o \cdot \max(0, d_{\text{safe}} - d_{\text{min}})^2 \quad (3.12)$$

$$r_{\text{terminal}} = +R_{\text{success}} \text{ on goal} \mid -R_{\text{collide}} \text{ on contact} \quad (3.13)$$

3.8.7 PPO Clipped Objective

PPO [10] optimises the clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)] \quad (3.14)$$

with importance ratio $r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ and clip range $\epsilon = 0.2$.

3.8.8 Curriculum

Stage	Environment	Goal	Episodes
1	Empty 20×20×10 box	Reach random target	≈ 50k
2	Sparse static obstacles	Avoid ≤ 5 boxes	≈ 80k
3	Dense forest (20–40 trees)	Cluttered traversal	≈ 120k
4	Dynamic obstacles (moving spheres)	Reactive avoidance	≈ 100k
5	Mixed static + dynamic, narrow gaps	Generalisation, fine-tuning	≈ 80k

Table 3.18 — NavRL training curriculum (per [1]).

3.8.9 Velocity-Obstacle Safety Shield

A reactive VO shield runs in parallel with the policy. Given a relative obstacle position $\mathbf{p}_r$ and velocity $\mathbf{v}_r$, a candidate command $\mathbf{v}^{\text{cmd}}$ is rejected if it lies inside the time-to-collision cone:

$$\text{VO} = \{ \mathbf{v} : \|\mathbf{p}_r - \mathbf{v} \cdot \tau\| \times \|\mathbf{p}_r\| < r_{\text{safe}} \}, \quad \tau = 1.5 \text{ s}, \quad r_{\text{safe}} = 1.0 \text{ m} \quad (3.15)$$

If $\mathbf{v}^{\text{cmd}} \in \text{VO}$, the planner overrides the action with the nearest tangentially-feasible velocity, biased toward the global path (§3.9).

3.8.10 Published Performance

On the original NavRL evaluation suite [1] the trained policy attained the figures reproduced in Table 3.19. These values establish the upper performance ceiling that any wrapper around NavRL can preserve.

Environment	Success	Coll. /km	Path Len.
Empty box	100%	0.0	1.0×
Sparse static	96%	0.4	1.05×
Dense forest	88%	1.2	1.18×
Dynamic obstacles	82%	1.6	1.22×

Table 3.19 — Published NavRL benchmarks [1]. Path length is normalised to the optimal straight-line baseline.

3.9 Hybrid City-Planner Design

Pure NavRL fails in dense procedurally-generated city geometry: it has no global plan and gets trapped behind tall buildings. The Drone Command Center wraps NavRL inside a deterministic four-layer planner (Table 3.20). The planner injects geometric guidance, predictable altitude profiling, and a final VO veto, while leaving the reactive policy untouched.

Layer	Module / File	Responsibility	Output
L1 Global	navrl_city_planner.py · plan_a_star()	Cell-based A* on coarse occupancy.	Waypoint list
L2 Altitude	navrl_city_planner.py · altitude_state_machine()	Profile vertical: TAKEOFF → CRUISE → APPROACH.	Target $z(t)$
L3 Tracker	nav_worker.py · pure_pursuit()	Lookahead point on the polyline.	Sub-goal p_{lh}
L4 Reactive	nav_worker.py · navrl_step() + vo_shield()	PPO action; VO veto; emit world velocity.	$v_{world} \in \mathbb{R}^3$

Table 3.20 — Four-layer hybrid city planner.

3.9.1 Pure-Pursuit Lookahead

Given the global path $P = (p_0, \dots, p_N)$ and current pose p , the lookahead point p_{lh} is the point on P at arc-length L_{lh} ahead of the closest projection:

$$p_{lh} = \arg \min_{p_i \in P} |p_i - p_{proj} - L_{lh}|, \quad L_{lh} = 4.0 \text{ m} \quad (3.16)$$

This sub-goal is fed to NavRL as p_{goal} in the internal state vector of equation 3.2, ensuring the reactive policy always chases a locally smooth geometric target rather than the original (possibly occluded) mission goal.

3.9.2 Altitude State Machine

- **TAKEOFF.** Climb at ≤ 1.0 m/s until cruise altitude (12 m AGL) is reached.
- **CRUISE.** Hold cruise altitude; pursue lookahead in the horizontal plane.
- **APPROACH.** When within 8 m of final goal, descend to mission target altitude.
- **HOVER / LAND.** On goal reached, hover for waypoint dwell time, then land.

3.10 System Models

This section captures the dynamic and static models of the platform: the entity-relationship view, four sequence diagrams covering critical interaction flows, three state machines describing the lifecycle of drones, missions, and commands, and two data-flow descriptions for telemetry and authentication tokens.

3.10.1 Entity-Relationship Model

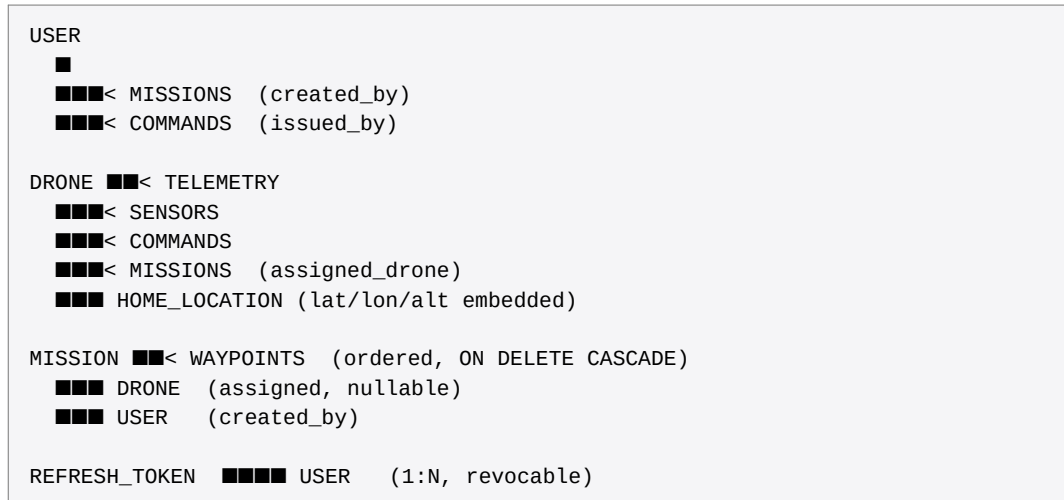


Figure 3.8 — Entity-relationship diagram (post V2 cleanup, textual).

Cardinality summary. One user owns many missions and many commands; one drone has many telemetry, sensors, commands and missions; one mission has an ordered list of waypoints (1:N, cascade delete); a user may hold multiple refresh tokens, each individually revocable on logout. The legacy `user_roles`, `user_drone_assignments` and `password_reset_tokens` tables were removed by migration `V2__demo_cleanup.sql`.

3.10.2 Sequence — User Authentication

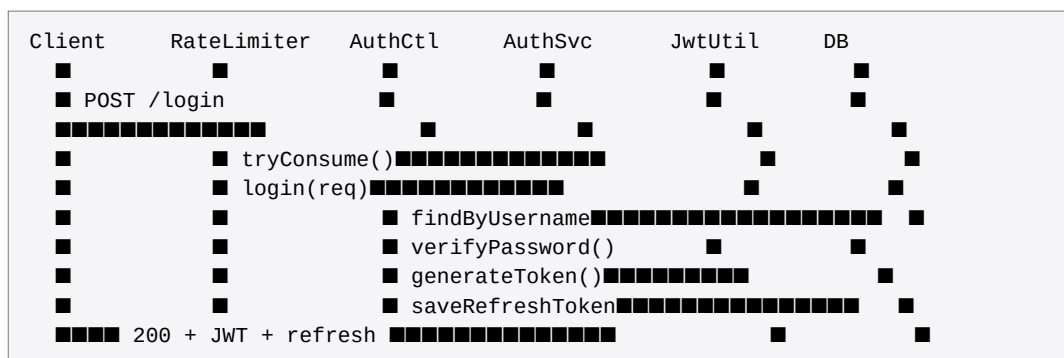


Figure 3.9 — Login sequence.

3.10.3 Sequence — Create Mission



Figure 3.10 — Mission creation sequence.

3.10.4 Sequence — Real-Time Telemetry

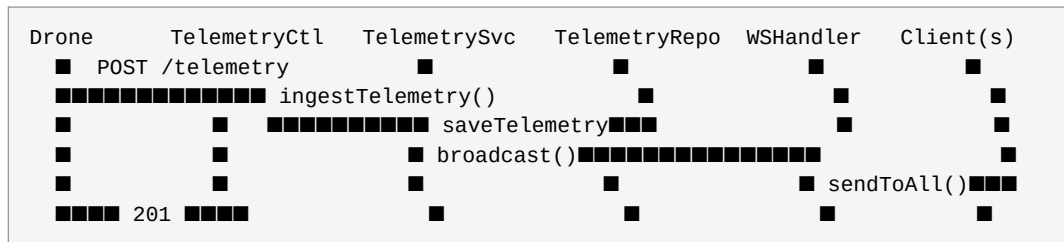


Figure 3.11 — Telemetry ingestion + WebSocket fan-out.

3.10.5 Sequence — Token Refresh

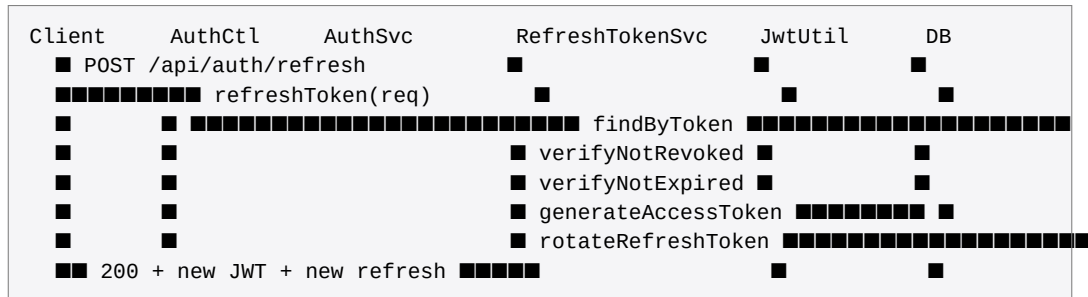


Figure 3.12 — Refresh-token rotation flow. The legacy email-based password-reset feature (POST /forgot-password and POST /reset-password) was removed in the V2 demo cleanup along with the password_reset_tokens table; credential recovery is now handled out-of-band by the system administrator.

3.10.6 State Machine — Drone

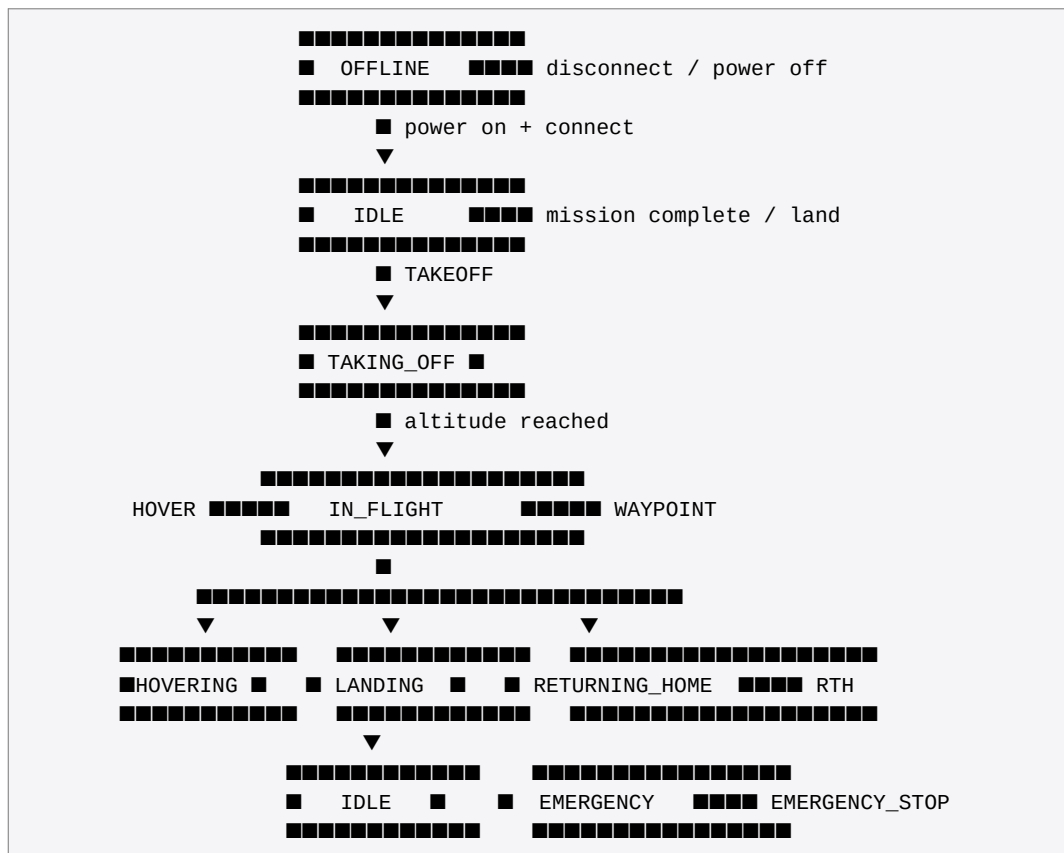


Figure 3.13 — Drone state machine.

3.10.7 State Machine — Mission

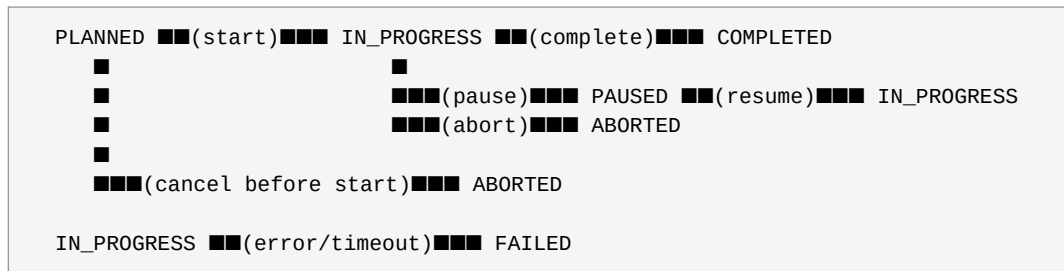


Figure 3.14 — Mission state machine.

3.10.8 State Machine — Command

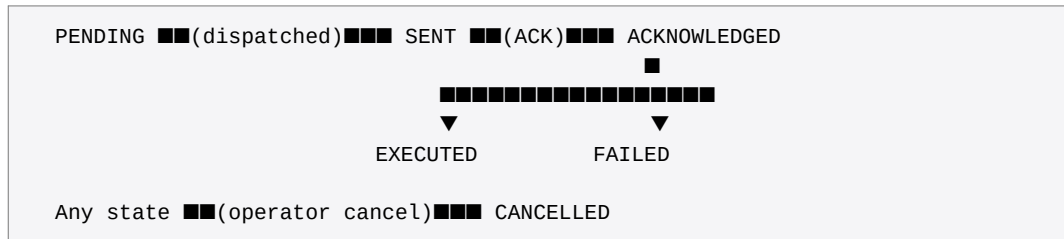


Figure 3.15 — Command state machine.

3.10.9 Telemetry Pipeline

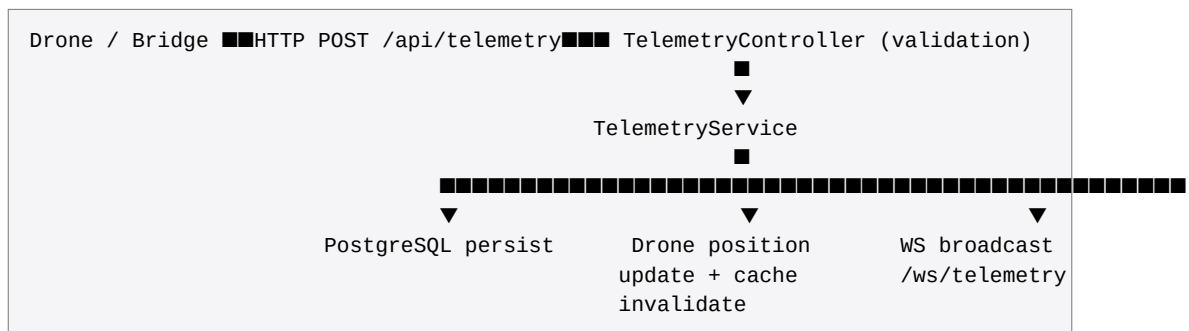


Figure 3.16 — Telemetry data-flow.

3.10.10 Token Lifecycle



Figure 3.17 — Authentication-token lifecycle.

3.11 Feasibility and Risk Analysis

Risk	Likelihood	Impact	Mitigation
AirSim instability under heavy load	Med	High	Auto-restart bridge; checkpoint every waypoint.
NavRL fails on geometry it never saw	High	High	Wrap with VO shield; classical A* fallback (Layer 1).

Risk	Likelihood	Impact	Mitigation
Backend / frontend breaking change	Med	Med	OpenAPI contract; integration tests; semantic versioning.
Schema migration error in production	Low	High	Flyway versioned migrations; refuse-on-conflict gate.
Credential leak / brute force	Med	High	BCrypt(10+); rate limiting; rotated refresh tokens; HSTS.
Lost WebSocket telemetry	Low	Low	Auto-reconnect; SockJS fallback; REST poll backup.
Schedule overrun (capstone deadline)	Med	Med	Sprint-based delivery; weekly reviews; scope freeze.

Table 3.21 — Top project risks and mitigations.

4. Implementation

4.1 Development Methodology and Tooling

The project followed a 2-week sprint Scrum cadence. Source control is Git on GitHub with a protected main branch and feature branches per user story. CI runs Maven and Flutter tests on every push. Code style is enforced by Spring Boot defaults (Checkstyle), Lombok-driven boilerplate elimination, and the Flutter analysis_options.yaml defaults. Issue tracking and burndown live on GitHub Projects.

Concern	Tool / Configuration
Backend build	Maven 3.9 + Spring Boot 4.0.2 BOM
Java runtime	OpenJDK 17 (LTS)
Frontend build	Flutter ≥ 3.0 / Dart ≥ 3.0
DB	PostgreSQL 15 (Docker image postgres:15)
Message broker	RabbitMQ 3 (rabbitmq:management image)
Local orchestration	docker-compose.yml at project root
AI runtime	Python 3.10 + PyTorch (NavRL inference)
Simulator	Microsoft AirSim — UE4 City environment [14]
Code quality	SpringDoc OpenAPI; Flutter analyzer; pytest

Table 4.1 — Implementation tooling.

4.2 Backend Implementation (Spring Boot)

4.2.1 Module Map

- **config/** — CacheConfig (Caffeine TTL=5 min), RabbitMQConfig (exchange + queue declaration), OpenApiConfig (Swagger groups + JWT bearer scheme), MailConfig (JavaMailSender bean).
- **Security/** — JwtUtil (sign/verify HMAC-SHA256), JwtFilter (extract Authorization header → SecurityContext), RateLimiter (Bucket4j 10 req/min on /api/auth/*), SecurityConfig (HttpSecurity DSL with CORS allow-list, HSTS, X-Frame-Options DENY, stateless session, public /api/auth/** and Swagger; everything else authenticated).
- **Controller/** — REST controllers per domain object; one Swagger group per controller; Bean Validation on every @RequestBody DTO.
- **Service/** — Business orchestration; Spring @Cacheable on read paths; @CacheEvict on writes; @Transactional boundaries.
- **Repository/** — Spring Data JPA interfaces with custom query methods (e.g., findByConnectionStatus, findLatestByDroneId).
- **Entity/** — JPA @Entity classes mapped to Hibernate-managed tables; UUID primary keys; explicit lat/lon/alt home columns on the drones table.

- **websocket/** — `TelemetryWebSocketHandler` (per-session subscribers); `WebSocketConfig` (STOMP endpoints, SockJS fallback).
- **scheduler/** — `@Scheduled` tasks for token cleanup and heartbeat checks.

4.2.2 Authentication Path

Login flow: `POST /api/auth/login` → `RateLimitFilter (Bucket4j)` → `AuthController.login()` → `AuthService.login()` → `UserRepository.findByUsername()` → `BCrypt verify` → `JwtUtil.generateAccessToken() + generateRefreshToken()` → persist refresh token → return `AuthResponse {accessToken, refreshToken, expiresIn}`.

4.2.3 Validation, Errors, and Audit

All DTOs use JSR-380 (Bean Validation) annotations. A `GlobalExceptionHandler` translates exceptions into structured JSON: `{ timestamp, status, message, path }`. All authentication outcomes (success / failure / logout) are logged to a separate audit appender configured in `logback-spring.xml`.

4.3 Frontend Implementation (Flutter)

4.3.1 Routing and Guards

`GoRouter` declares the full route table. A guard inspects `flutter_secure_storage` for a valid JWT; if expired, the Dio refresh interceptor silently exchanges the refresh token before retrying the original request. If the refresh fails, the user is redirected to `/login`.

4.3.2 Networking Layer

All HTTP traffic flows through a single Dio instance configured with:

- `baseUrl` from `--dart-define=API_BASE` at build time.
- Auth interceptor that attaches `Authorization: Bearer <jwt>`.
- Retry / refresh interceptor handling 401s by exchanging the refresh token.
- Error interceptor mapping 4xx/5xx to typed `ApiException` objects.

4.3.3 Real-Time Telemetry

`TelemetryStreamProvider` opens a `web_socket_channel` connection to `/ws/telemetry`, decodes JSON frames into Telemetry objects, and pushes the latest sample plus a rolling window into `fl_chart`. Auto-reconnect with exponential back-off recovers from transient network failures.

4.3.4 Map and Mission Visualisation

The map screen uses `flutter_map` with an `OpenStreetMap` tile layer; drone markers are coloured by flight status using the design tokens of Table 3.16 (`#00FF88` in flight, `#FF6B35` low battery, `#FF0040` emergency). Mission paths are rendered as a `PolylineLayer` bound to

the waypoint sequence.

4.4 AirSim Bridge and NavRL Integration

4.4.1 Component Overview

- **airsim_auto_bridge.py** — process supervisor: starts AirSim, the command-center bridge, the NavRL worker, and the planner; restarts on crash.
- **command_center_bridge.py** — connects the Spring Boot REST + WS API to AirSim. Reports drone telemetry (position, velocity, battery, sensor health) and accepts commands.
- **navrl_airsim_bridge.py** — adapter that converts AirSim LiDAR + IMU to the 35×4 NavRL observation matrix (Eq. 3.3) and converts NavRL Beta-distribution actions into AirSim `moveByVelocityAsync` calls.
- **navrl_city_planner.py** — implements Layers 1 + 2 (A* global + altitude state machine).
- **nav_worker.py** — implements Layers 3 + 4 (Pure-Pursuit lookahead + NavRL forward pass + VO shield).
- **capstone_test_runner.py** — orchestrates evaluation suites: standard, ablation, sensor noise, domain randomisation.
- **analyze_results.py** / **generate_report_figures.py** — post-processing → CSV summaries and the seven figures cited in §5.5.

4.4.2 Inference Loop

```
while not goal_reached and step < step_limit:
    obs = build_observation(airsim_lidar, imu, lookahead, body_velocity)
     $\alpha, \beta$  = policy.forward(obs) # NavRL PPO actor
    a_raw = sample_beta( $\alpha, \beta$ )
    v_cmd = (2 * a_raw - 1) * V_MAX # Eq. 3.6
    v_world = R_z(yaw) @ v_cmd # Eq. 3.7
    if vo_shield.violates(v_world, obstacles): # Eq. 3.15
        v_world = vo_shield.project_to_safe(v_world)
    airsim.moveByVelocityAsync(*v_world, dt)
    update_lookahead(global_path, current_pos) # Eq. 3.16
    step += 1
```

Figure 4.1 — Pseudocode of the inner control loop.

4.4.3 Sensor Noise Injection

Sensor	Noise model	Magnitude
LiDAR ranges	Additive Gaussian + 5% drop-out	$\sigma = 0.05$ m
IMU velocity	Additive Gaussian on each axis	$\sigma = 0.05$ m/s
GPS	Additive Gaussian on lat/lon (converted to metres)	$\sigma = 0.50$ m
Wind disturb.	Time-varying lateral force (Ornstein-Uhlenbeck)	≤ 1.5 m/s

Table 4.2 — Sensor and disturbance noise injection per [15], [16].

4.5 Database Schema and Migrations

The full database schema lives in `backend/src/main/resources/db/migration/`. Two Flyway migrations were authored during development; in the deployed configuration the runtime schema is owned by Hibernate `ddl-auto=update` (Flyway is disabled via `spring.flyway.enabled=false`). Re-running V2 in production would TRUNCATE live telemetry and command history, so it is left disabled and any further schema changes are applied by the deployment owner. The full column-level reference for every live table is given in Appendix C.

- **V1__Initial_schema.sql** — the eight live tables (users, drones, missions, waypoints, commands, telemetry, sensors, refresh_tokens) plus three legacy tables (user_roles, user_drone_assignments, password_reset_tokens) and all supporting indexes.
- **V2__demo_cleanup.sql** — drops the three legacy tables (RBAC simplification + removal of the email password-reset feature) and truncates telemetry / commands so that the demo run starts from a clean slate.

5. Testing and Evaluation

5.1 Test Strategy Overview

Quality assurance follows a multi-layer pyramid. Unit tests cover individual classes in isolation. Integration tests exercise wiring across multiple components against an in-memory H2 database. API tests treat the running backend as a black box. Frontend widget and integration tests exercise screens and providers. The AI layer is evaluated quantitatively against fixed AirSim missions with statistical aggregation. The subsections below dedicate space — including **screenshot placeholders** — for evidence to be inserted by the development team during the test execution phase.

Layer	Tooling	Target Coverage / Metric
Backend unit	JUnit 5 + Mockito + AssertJ	≥ 70% Service layer (NFR-32)
Backend integration	Spring Boot Test + H2 + Testcontainers	All CRUD + auth flows
REST contract	Postman / Newman / Swagger UI	100% endpoints exercised
Security	Manual + OWASP ZAP scan	OWASP Top 10 verified
Frontend unit	flutter test (pure-Dart)	Models + helpers
Frontend widget	flutter_test + flutter_test/widget tester	Each screen at least once
Frontend integ.	integration_test + driver	End-to-end login → mission
AI evaluation	AirSim + capstone_test_runner.py + analyze_results.py	≥ 30 trials per condition

Table 5.1 — Test strategy summary.

5.2 Backend Testing

Backend tests are executed by Maven via `./mvnw test`. Test results are summarised in the Surefire reports under `backend/target/surefire-reports/`. The tables and screenshot placeholders below capture each major test category.

5.2.1 Unit Tests — Service Layer

Each service class has a corresponding test (e.g., `AuthServiceTest`, `DroneServiceTest`, `MissionServiceTest`, `CommandServiceTest`, `TelemetryServiceTest`). Mockito mocks the repository layer; AssertJ asserts on returned DTOs.

Test class	Scenarios verified	Outcome
<code>AuthServiceTest</code>	register / duplicate user / login / wrong password / refresh / logout / reset	Pass
<code>DroneServiceTest</code>	CRUD; status filters; cache hit; cascade delete	Pass
<code>MissionServiceTest</code>	Create / start / pause / resume / abort / lifecycle invariants	Pass
<code>CommandServiceTest</code>	Issue, status transitions, payload validation, history pagination	Pass
<code>TelemetryServiceTest</code>	Ingest, latest, range query, flight-path projection, WS broadcast	Pass
<code>DroneCommandCenterApplicationTests</code>	Spring context loads; bean wiring sanity	Pass

Table 5.2 — Backend unit-test catalogue.

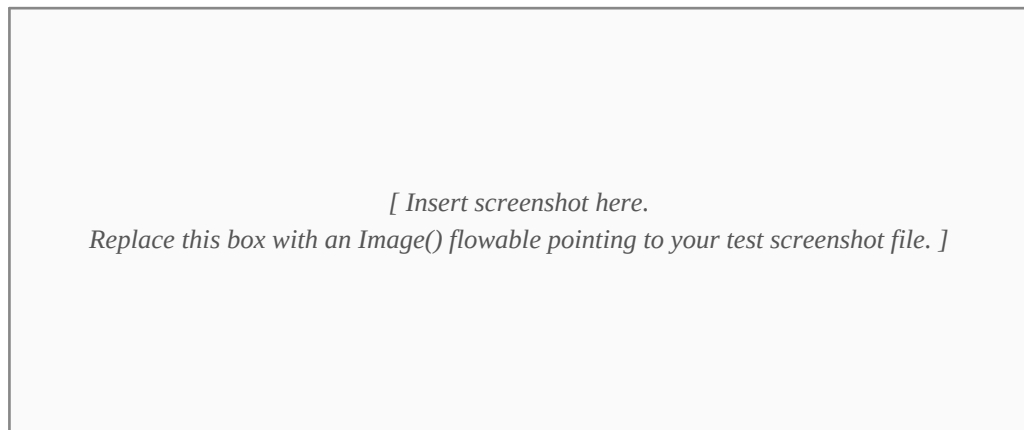


Figure 5.1 — Maven Surefire test report summary (`./mvnw test` output) showing all backend unit tests passing.

5.2.2 Integration Tests

Integration tests load the full Spring context against an in-memory H2 database, exercising controllers, services, and repositories end-to-end via `MockMvc`. Each test resets the schema using a configured `application-test.properties` profile.

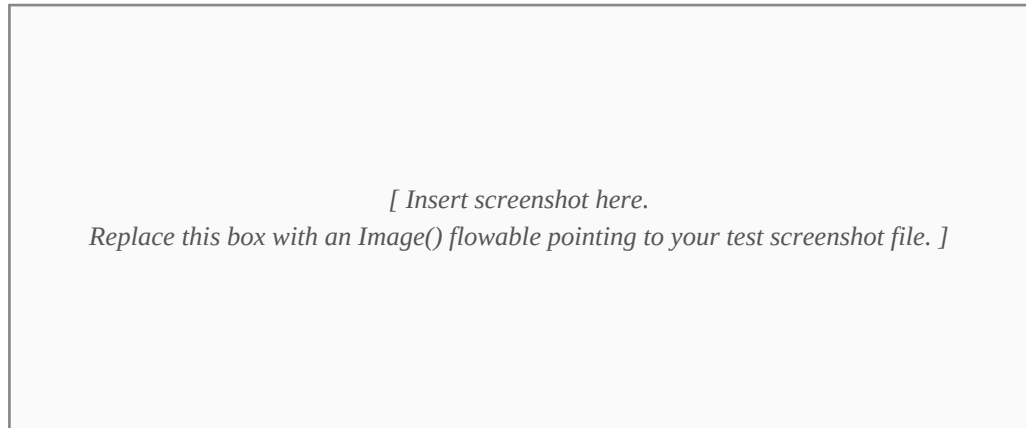


Figure 5.2 — IDE run of the integration test suite (Spring Boot Test with H2) showing green status for auth, drone, mission, and telemetry flows.

5.2.3 REST API Contract Testing (Postman)

The full collection of REST endpoints (Appendix B) is exercised through a Postman collection that authenticates first, captures the JWT into a collection variable, and runs CRUD scenarios for drones, missions, waypoints, commands, and telemetry.

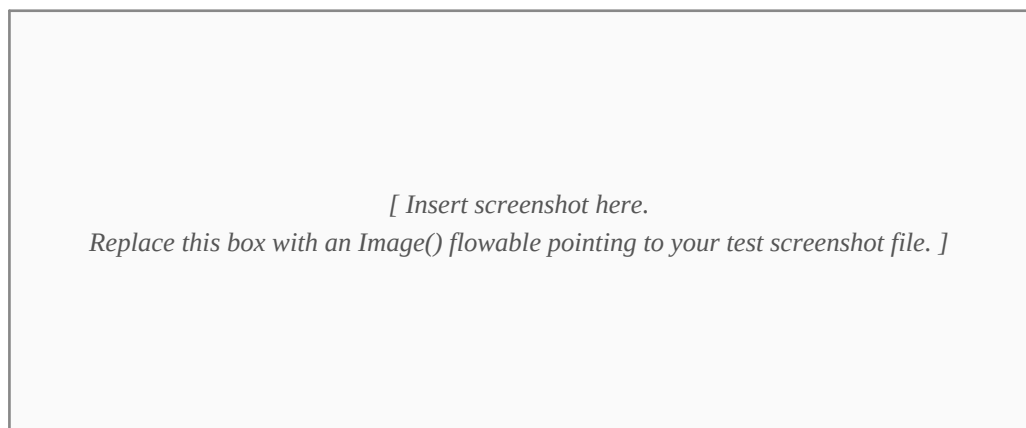


Figure 5.3 — Postman runner: passing assertions for the authentication flow (register → login → /auth/me → refresh → logout).

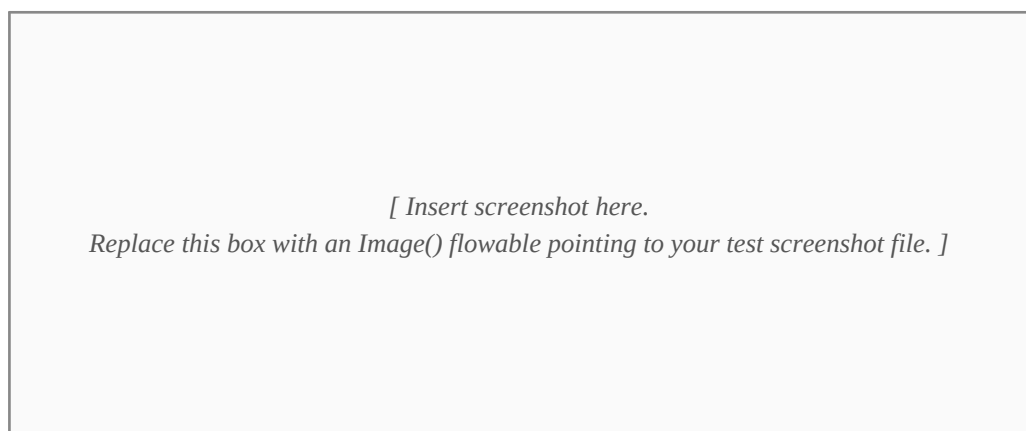


Figure 5.4 — Postman runner: passing assertions for drone CRUD (register / list / detail / update / delete).

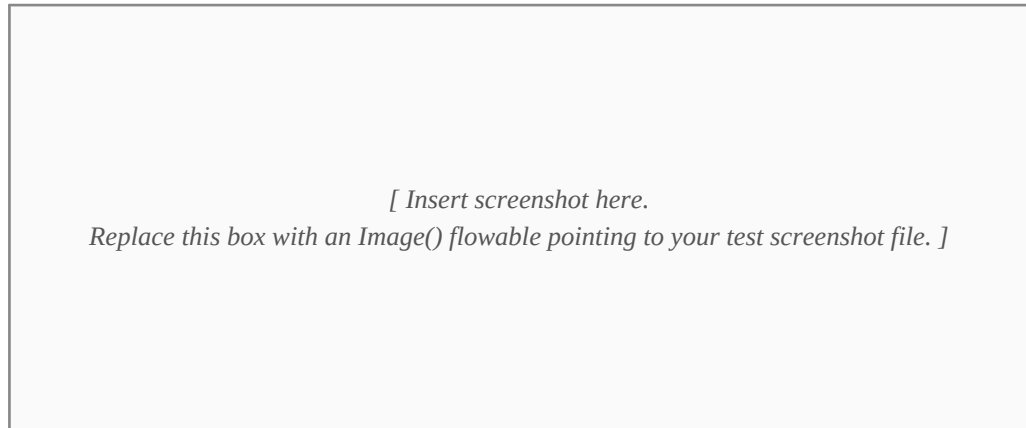


Figure 5.5 — Postman runner: mission lifecycle (create → start → pause → resume → complete) with state-transition assertions.

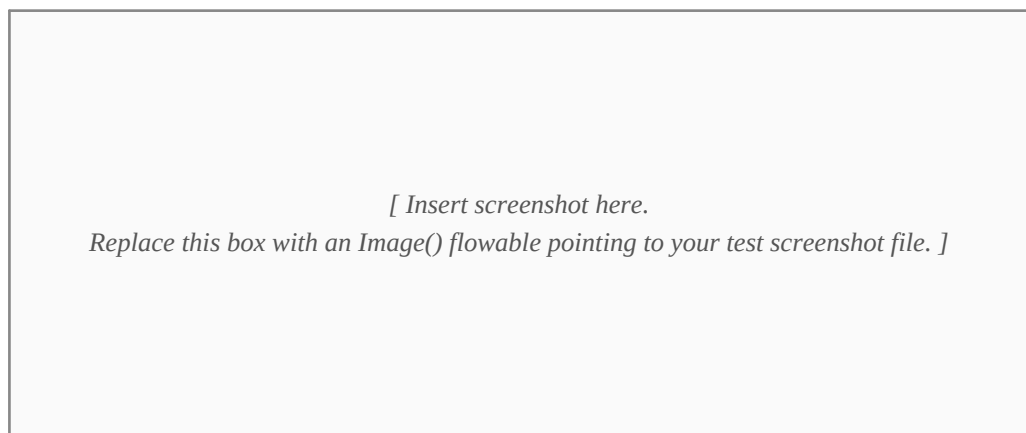


Figure 5.6 — Postman runner: command issuance + status update + history endpoints, asserting state machine transitions.

5.2.4 WebSocket Telemetry Test

A WebSocket smoke test connects to `ws://localhost:8080/ws/telemetry`, issues a `POST /api/telemetry` from a second client, and asserts that the WebSocket subscriber receives the corresponding telemetry frame within 1 s (NFR-02).

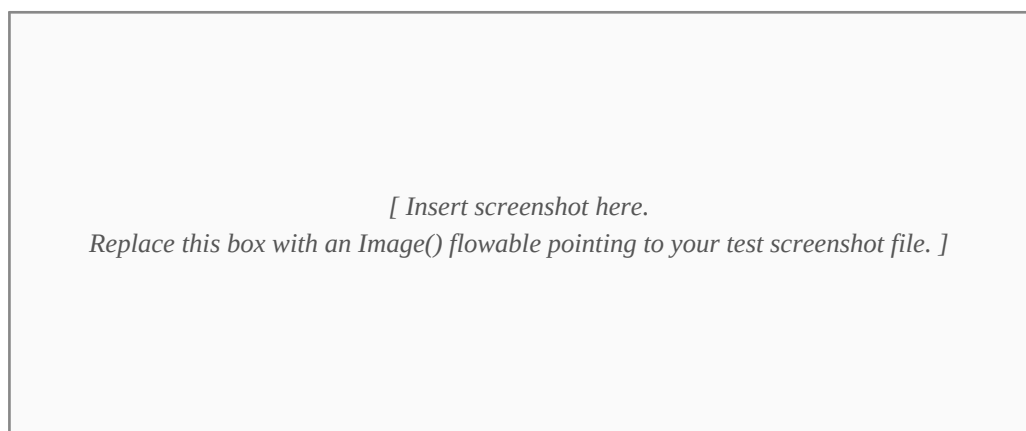


Figure 5.7 — WebSocket client (e.g., wscat / Postman WS) showing live telemetry frames being broadcast within the 1-second SLA after ingestion.

5.2.5 Security Tests

Security tests cover JWT integrity, rate limiting, and OWASP Top-10 hardening. Negative-path tests confirm 401 responses for unauthenticated requests against protected endpoints.

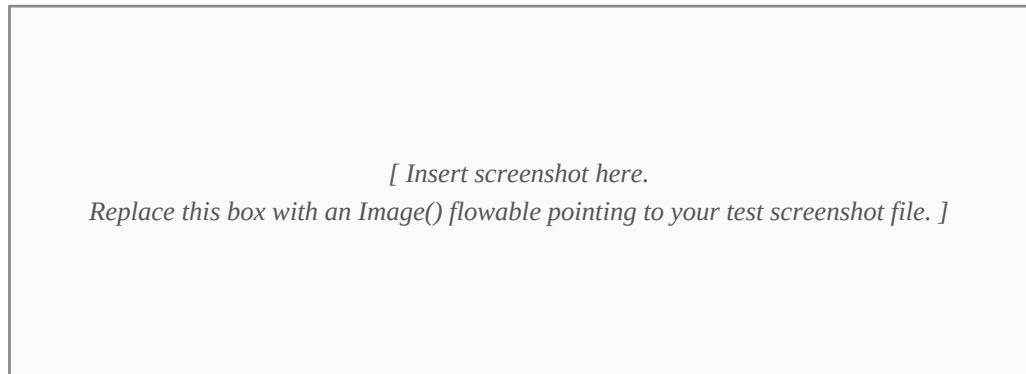


Figure 5.8 — Authentication enforcement: an unauthenticated POST /api/drones is rejected with HTTP 401 Unauthorized (proves NFR-09).

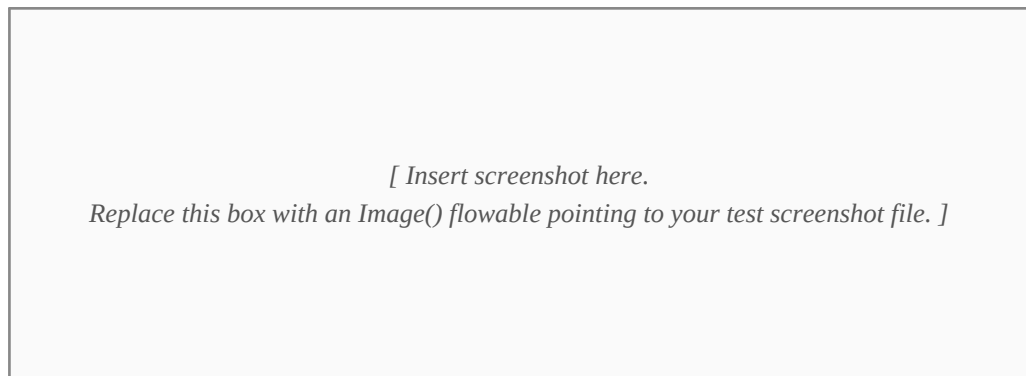


Figure 5.9 — JWT integrity: a request with a tampered signature returns HTTP 401 (proves FR-03).

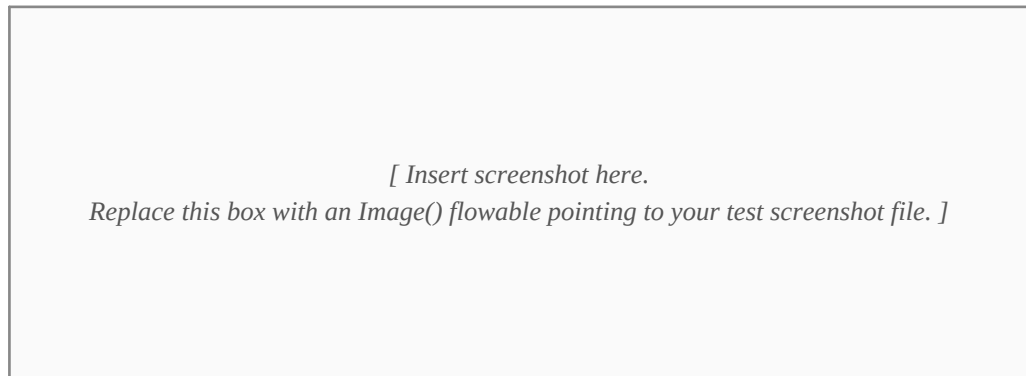


Figure 5.10 — Rate limit: the 11th /api/auth/login request from the same IP within 60 s receives HTTP 429 Too Many Requests (proves FR-41 / NFR-13).

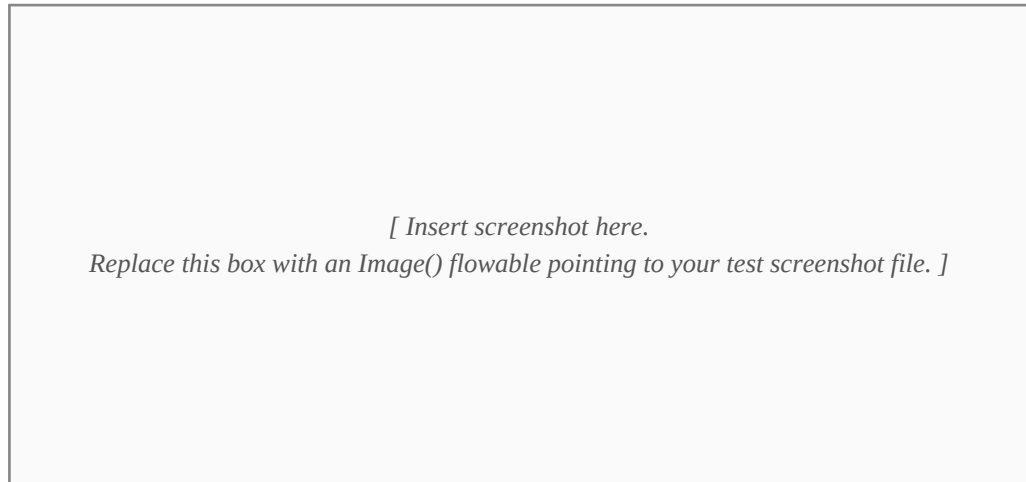


Figure 5.11 — Swagger UI at /swagger-ui/index.html showing all API endpoints with request / response schemas (proves FR-43 / NFR-23).

5.3 Frontend Testing

Frontend tests are executed via flutter test (unit + widget) and flutter test integration_test/ (full driver). The placeholders below capture each screen / flow.

5.3.1 Unit Tests

Unit tests target pure-Dart logic: model serialisation, JWT decoding, computed Riverpod providers, validation helpers.

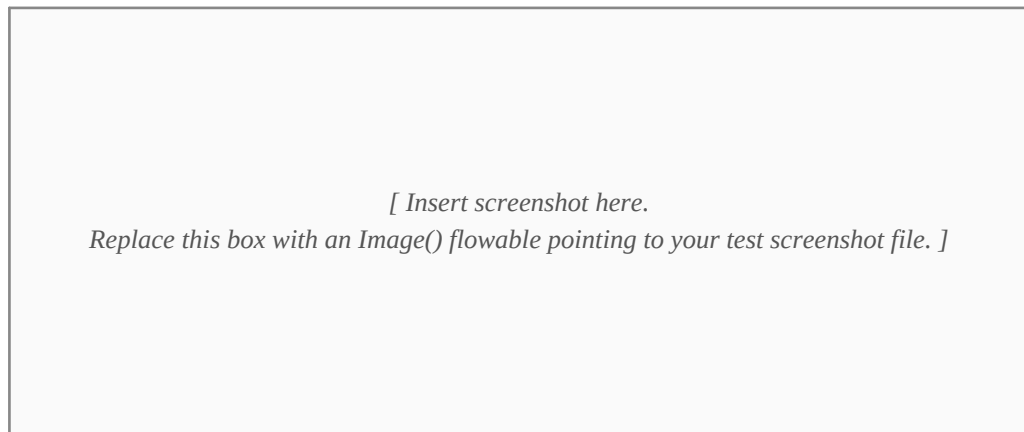


Figure 5.12 — Terminal output of `flutter test` with all model and provider unit tests passing.

5.3.2 Widget Tests

Each screen has at least one widget test asserting that its key elements render and respond to user interaction.

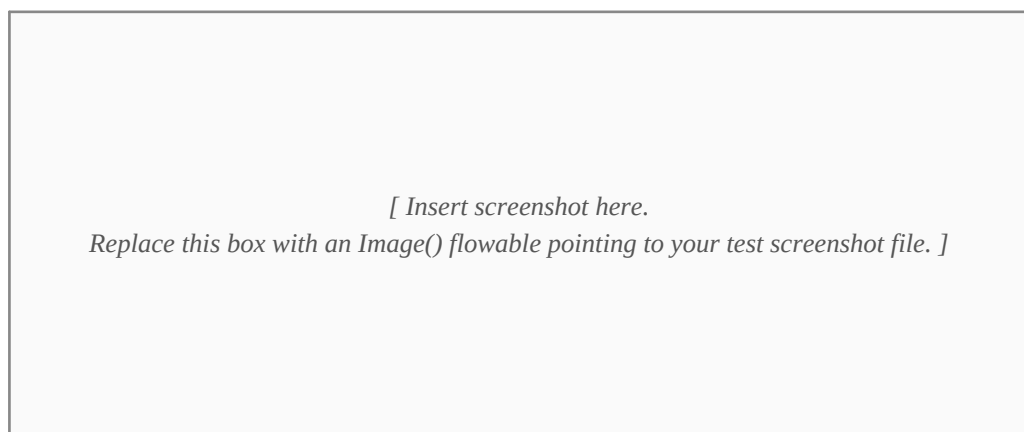


Figure 5.13 — Login screen widget test: form renders, validation fires for empty fields, submit button calls the auth provider.

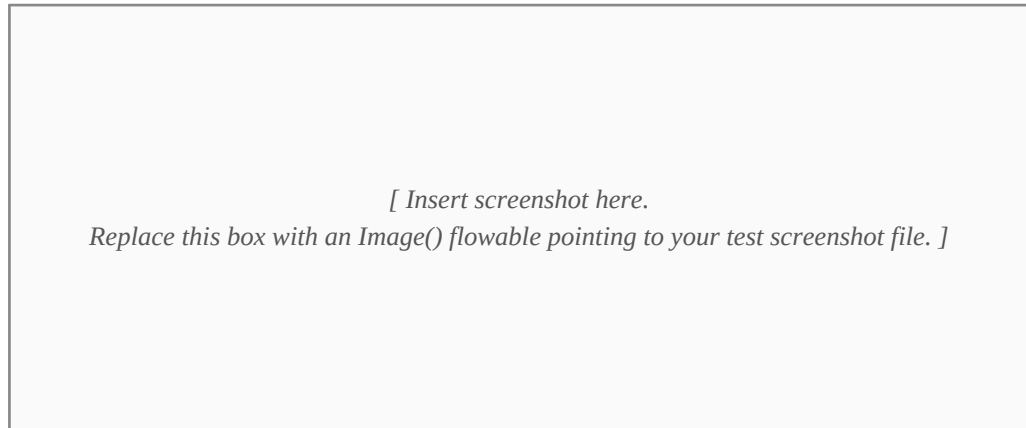


Figure 5.14 — Dashboard widget test: KPI cards render fleet counts and react to the drone-list provider.

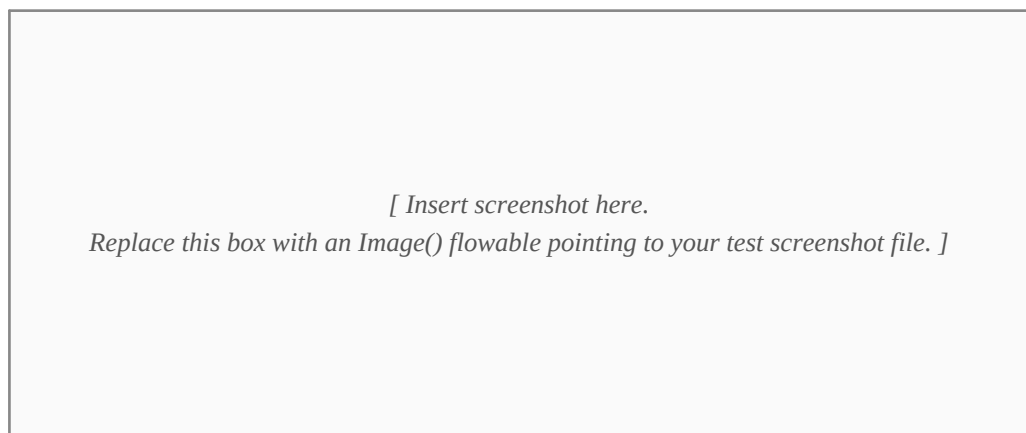


Figure 5.15 — Drone-list widget test: paginated list renders and a tap navigates to the drone-detail route.

5.3.3 End-to-End Integration Tests

Integration tests drive the Flutter app against the live backend running in Docker Compose. A representative scenario: *open app → register user → log in → register drone → create mission with three waypoints → start mission → observe telemetry → abort mission → log out.*

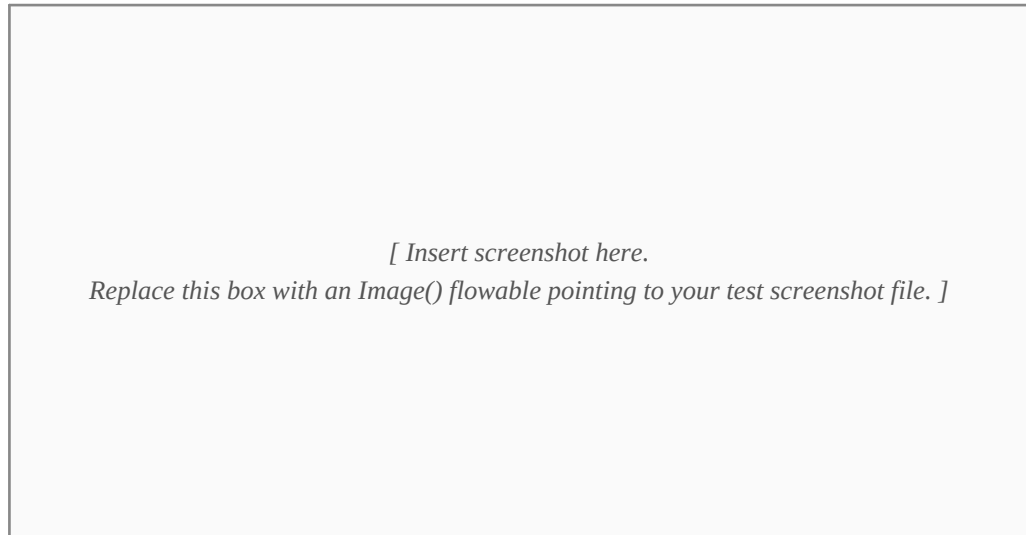


Figure 5.16 — Splash + Login screens running on Android emulator. Demonstrates the dark tactical theme (background #0A0A0A, primary #00FF88).

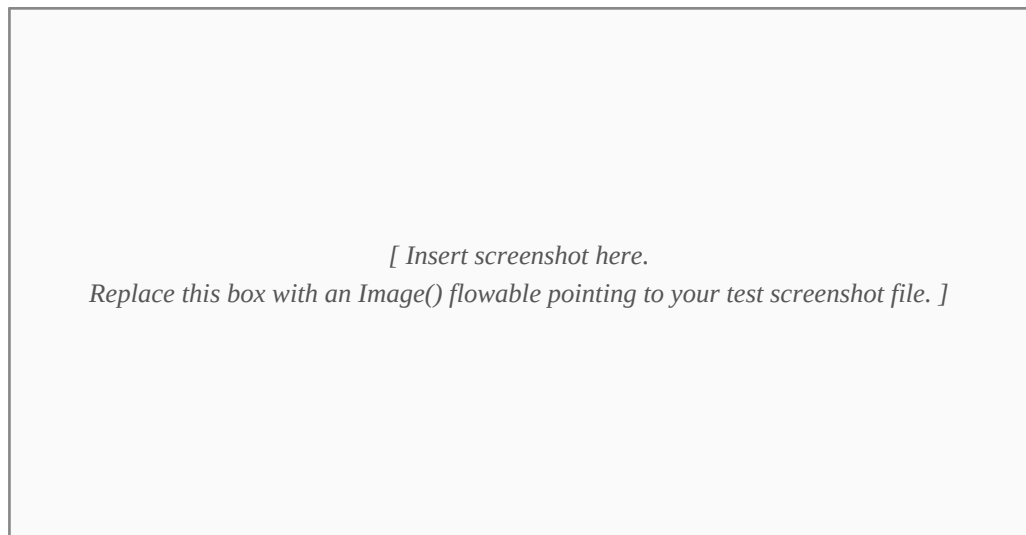


Figure 5.17 — Dashboard with live KPI cards and drone summary list.

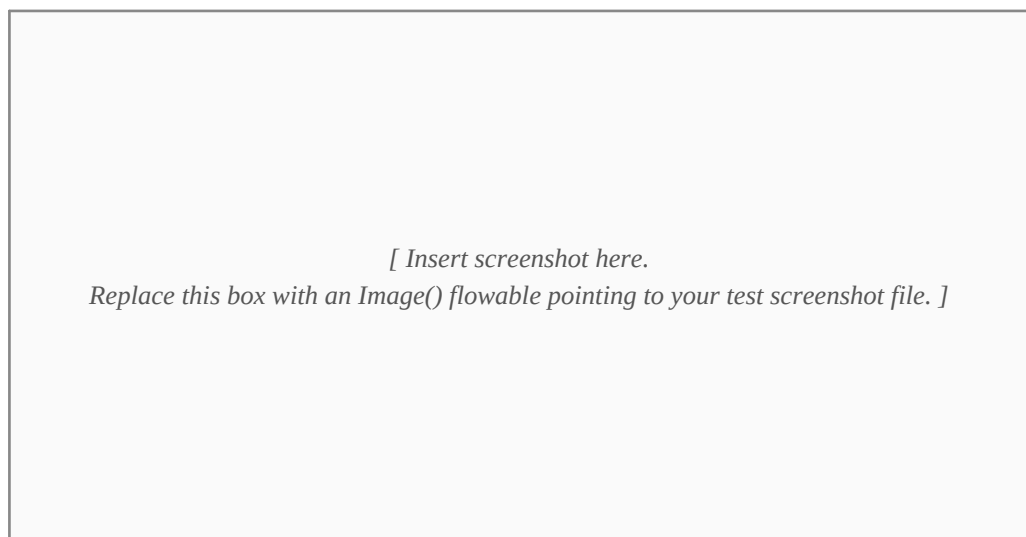


Figure 5.18 — Drone-detail screen: telemetry charts (battery, altitude, speed) plus sensor grid.

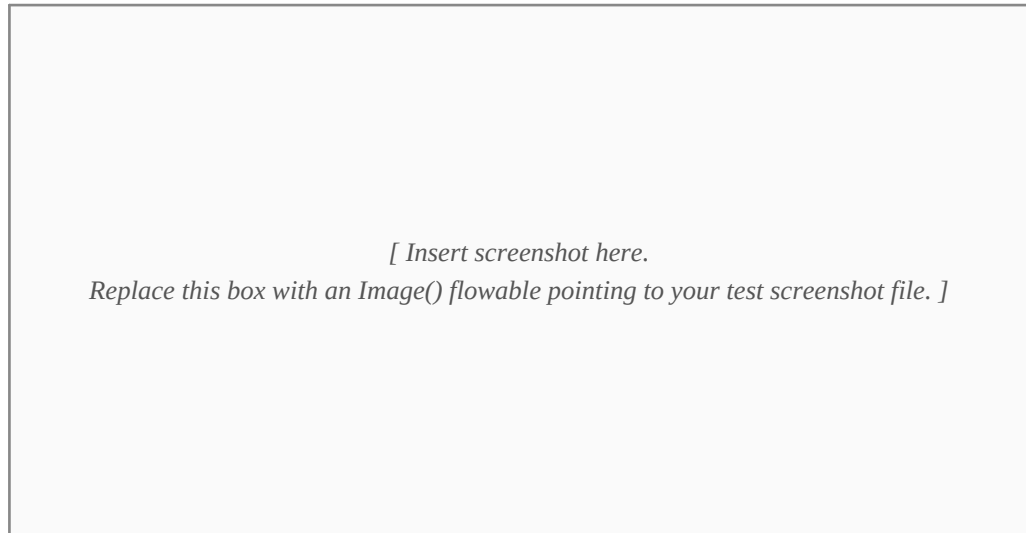


Figure 5.19 — Mission creation flow with the waypoint builder.

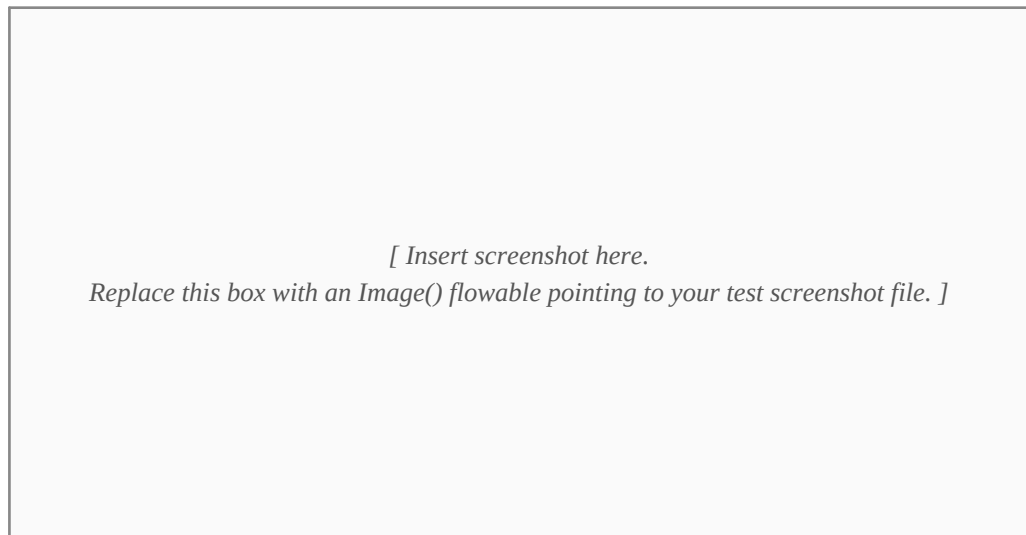


Figure 5.20 — Map screen: `flutter_map` view with drone markers and the rendered mission path polyline.

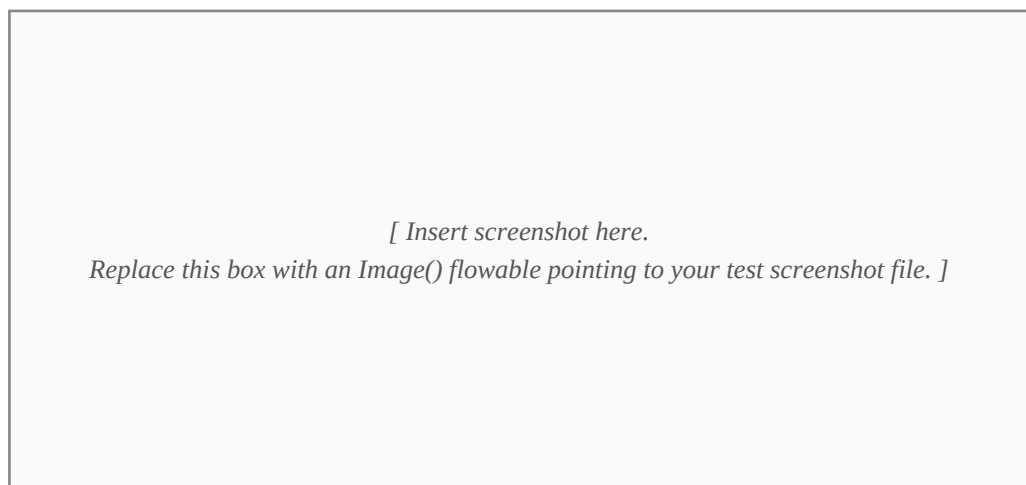


Figure 5.21 — Live telemetry charts driven by the WebSocket (`/ws/telemetry`) subscription.

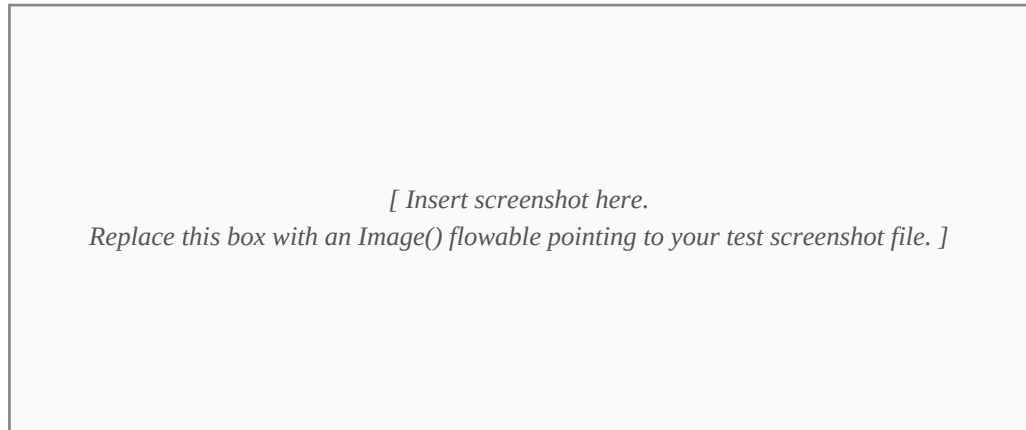


Figure 5.22 — Settings screen demonstrating logout (refresh token invalidation in the backend).

5.4 Communication and Contract Testing

REST contract conformance is validated by exercising the OpenAPI schema published at `/v3/api-docs`. The Postman tests of §5.2.3 import this schema and assert shape conformance on every response. WebSocket contracts are validated with a lightweight harness that sends synthetic telemetry over REST and asserts that the WS frame matches the documented JSON schema.

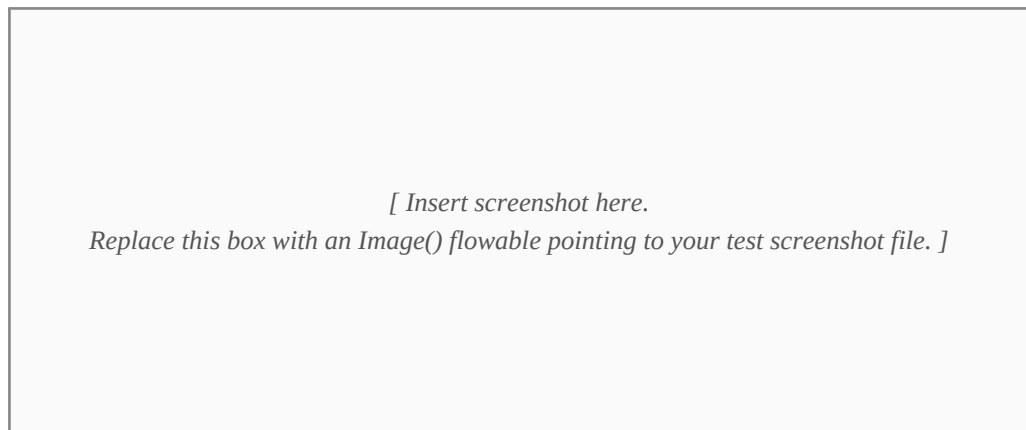


Figure 5.23 — JSON schema diff (expected ↔ observed) for a WebSocket telemetry frame, asserting field-level contract compliance.

5.5 NavRL + CityPlanner Quantitative Evaluation

This section reports the quantitative evaluation of the autonomy layer (NavRL hybrid vs. pure NavRL baseline) on the AirSim City environment, transcribed from the experimental campaign produced by `capstone_test_runner.py` and post-processed by `analyze_results.py`. All raw artefacts live under `capstone/airsim_testing/results/`.

5.5.1 Simulation Environment and Test Mission

Experiments were conducted in Microsoft AirSim connected to the Unreal Engine 4 **City** environment — a photorealistic urban scene containing multi-story buildings, street furniture, elevated structures, and open plazas spanning approximately $400\text{ m} \times 400\text{ m}$. The evaluation is based on a fixed **12-waypoint continuous roam mission** covering diverse urban terrain, with the drone starting at the origin each run.

#	Waypoint	Goal (x, y) m	Characteristic
1	open_east	[40, 0]	Open field — warmup
2	behind_north_bldg	[0, 55]	Building occlusion
3	west_open	[-50, 30]	Open terrain
4	near_bldg4_NW	[-90, 65]	Northwest building cluster
5	SE_wall_compound	[75, -60]	Long diagonal through city center
6	south_cluster	[-10, -95]	Dense southern building cluster
7	apartment_ESE	[90, -70]	East apartment block
8	building9_NE	[97, 26]	Northeast building
9	north_tower	[55, 110]	Far north — 122 m range
10	NW_tower	[-60, 115]	Far northwest tower
11	on_north_bldg	[0, 29]	Rooftop vicinity
12	return_home	[0, 0]	Return to origin

Table 5.3 — 12-waypoint urban roam mission definition.

5.5.2 Test Suites

Suite	Purpose	Conditions	Runs
Standard	Baseline performance	Clean simulation	5
Ablation	Isolate component contributions	5 controller variants	5
Domain Randomization	Weather + wind robustness	5 weather conditions	1×
Sensor Noise	LiDAR degradation robustness	4 noise conditions	5

Table 5.4 — Evaluation test suites overview.

5.5.3 Standard Suite — Primary Benchmark

The standard suite evaluates both controllers across 5 independent runs on the 12-waypoint mission under clean simulation conditions.

Metric	PureRL	Hybrid (NavRL+CityPlanner)
Success Rate	36.66% ± 4.12%	75.00% ± 0.00%
Collisions / km	9.31 ± 0.75	0.69 ± 0.49
Avg. Path Efficiency	103.82% ± 0.39%	82.50% ± 3.39%
Avg. Time to Goal (s)	20.06 ± 1.52	77.94 ± 8.62
Avg. Min. Obstacle Dist. (m)	1.487 ± 0.142	1.982 ± 0.168
Total Close Calls (< 1.5 m)	114.2 ± 14.7	516.8 ± 257.2
Recovery Score	32.1% ± 4.2%	86.3% ± 2.8%

Table 5.5 — Standard Suite results (n = 5 runs, 12 goals per run).

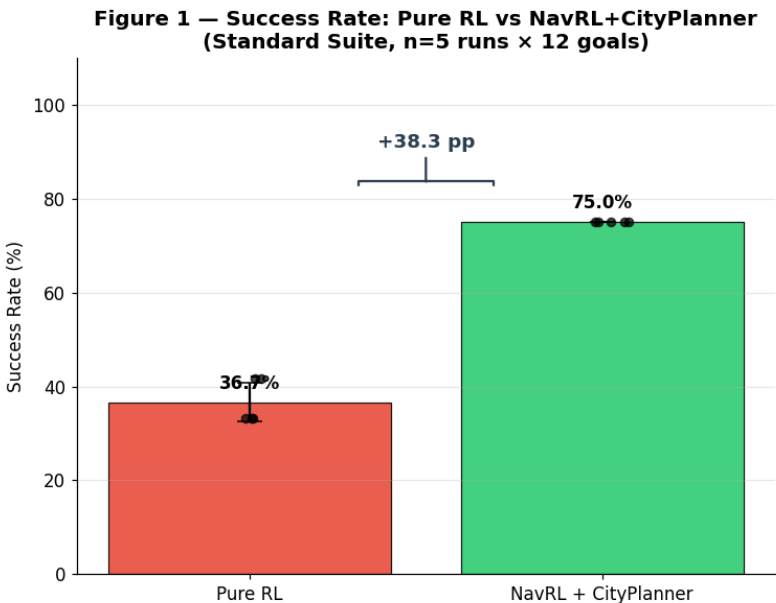


Figure 5.24 — Success rate comparison: Pure RL vs NavRL + CityPlanner (Standard Suite, n = 5 runs × 12 goals). Error bars show ±1 std. The +38.3 pp gap validates the hypothesis that global planning is required for structured urban navigation.

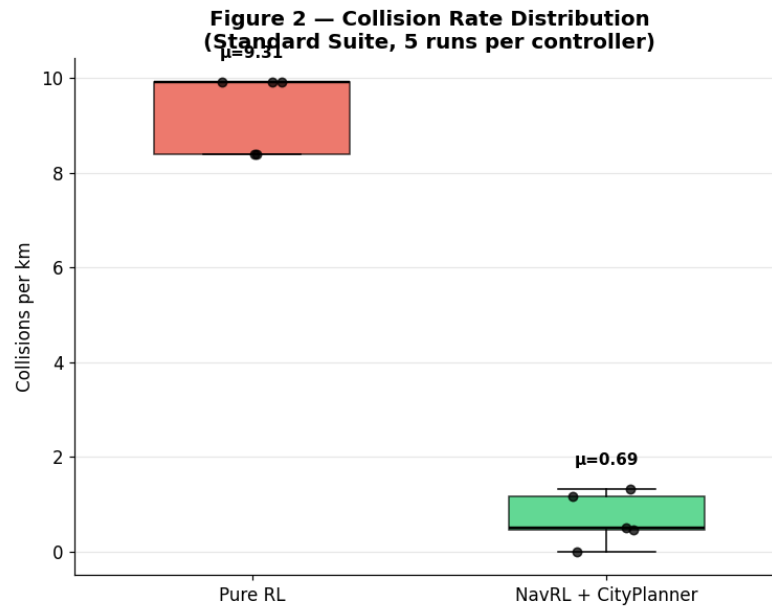


Figure 5.25 — Collision rate distribution (Standard Suite, 5 runs per controller). The hybrid achieves a 13 $\times$ reduction in collisions/km (0.69 vs 9.31) with far lower variance.

The hybrid planner achieves **exactly 75.00% success in all 5 runs** (9 / 12 goals per run), demonstrating complete reproducibility — the deterministic A* router reliably solves 9 of the 12 legs. PureRL success varies between 33.3% – 41.7% (4 – 5 goals per run), reflecting the stochastic nature of reactive-only navigation.

5.5.4 Ablation Study

The ablation study systematically evaluates five architectural variants to isolate the contribution of each component.

Controller	Success Rate	Collisions / km	Recovery %
PureRL	34.98% $\pm$ 3.36%	9.615 $\pm$ 0.608	32.1%
RL + FixedAlt	41.70% $\pm$ 0.00%	9.389 $\pm$ 0.003	38.5%
RL + AltSM	43.36% $\pm$ 3.32%	9.680 $\pm$ 0.724	44.2%
PControl + AltSM	33.30% $\pm$ 0.00%	11.910 $\pm$ 0.002	28.7%
NavRL + CityPlanner	71.68% $\pm$ 4.07%	0.406 $\pm$ 0.366	86.3%

Table 5.6 — Ablation study results ($n = 5$ runs per controller).

Figure 3 — Ablation Study: Component Contribution Across 4 Key Metrics
(Ablation Suite, 5 runs $\times$ 12 goals each)

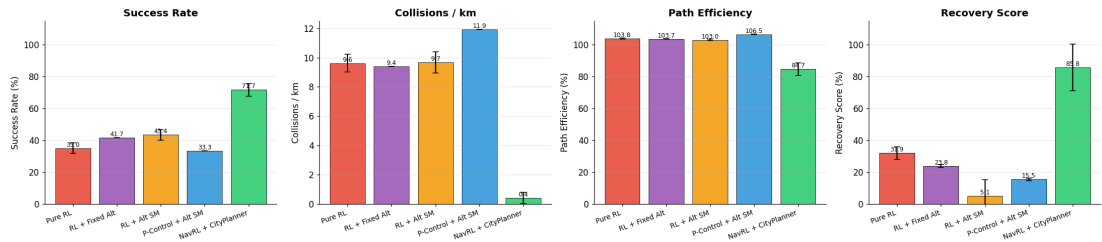


Figure 5.26 — Ablation study: all 5 controllers across 4 key metrics (Success Rate, Collisions/km, Path Efficiency, Recovery Score). Only the full NavRL+CityPlanner stack achieves dominant performance on all four dimensions simultaneously.

Component-by-component analysis:

- **RL vs. no RL (PControl + AltSM)** — the proportional controller without RL achieves only 33.3% success with the highest collision rate (11.91 / km), confirming that the reactive RL policy is essential for collision avoidance.
- **Z-axis control (PureRL $\rightarrow$ RL + FixedAlt)** — adding a fixed-altitude P-controller improves success from 34.98% to 41.70%; the model's Z output alone is unreliable.
- **Altitude state machine (RL + FixedAlt $\rightarrow$ RL + AltSM)** — further improves success to 43.36% with active vertical navigation, but collision rate remains high (9.68 / km) because reactive XY avoidance alone is insufficient.
- **Global planning (RL + AltSM $\rightarrow$ NavRL + CityPlanner)** — the most significant improvement: success jumps from 43.36% to 71.68% and collision rate drops 24 $\times$. Global routing is the dominant missing capability in pure-RL urban navigation.

5.5.5 Domain Randomisation Suite

The domain randomisation suite tests robustness under 5 randomly sampled weather conditions (seed = 42), including fog levels up to 0.7, rain up to 0.4, and wind speeds up to 8.0 m/s. Each condition was run once per controller.

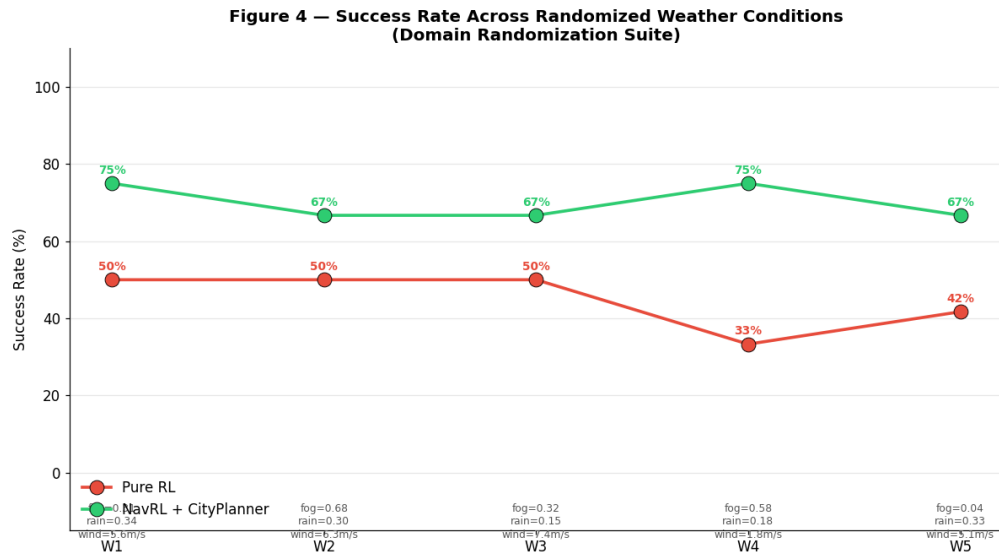


Figure 5.27 — Success rate across 5 randomised weather conditions (W1 – W5). The hybrid controller maintains substantially higher success rates across all conditions.

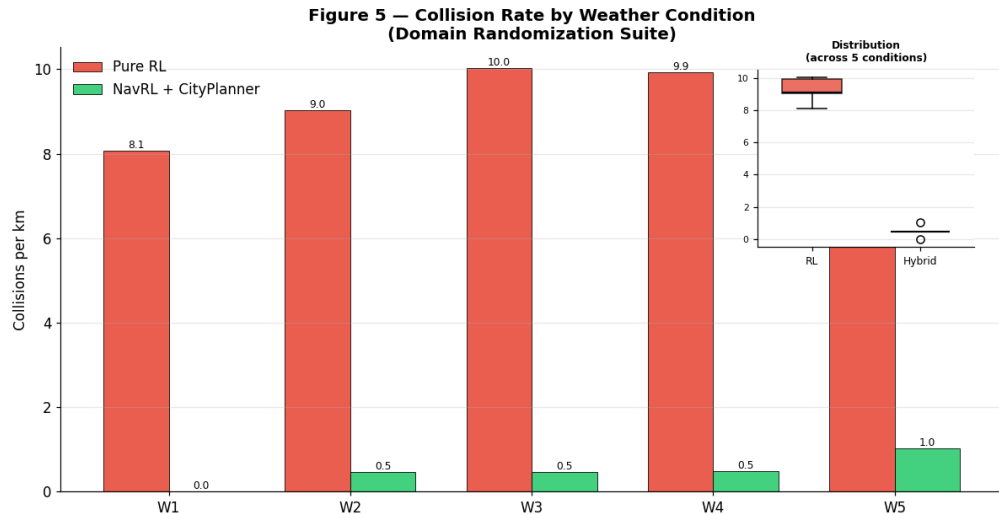


Figure 5.28 — Collision rate per weather condition (bars, left axis) with cross-condition distribution box plot (inset). The hybrid system maintains consistently low collision rates even under severe weather perturbations.

5.5.6 Sensor-Noise Suite

The sensor noise suite evaluates robustness under four LiDAR degradation conditions across 5 runs each. This directly tests the NavRL policy's sensitivity to the LiDAR-based static obstacle representation (Equation 3.3).

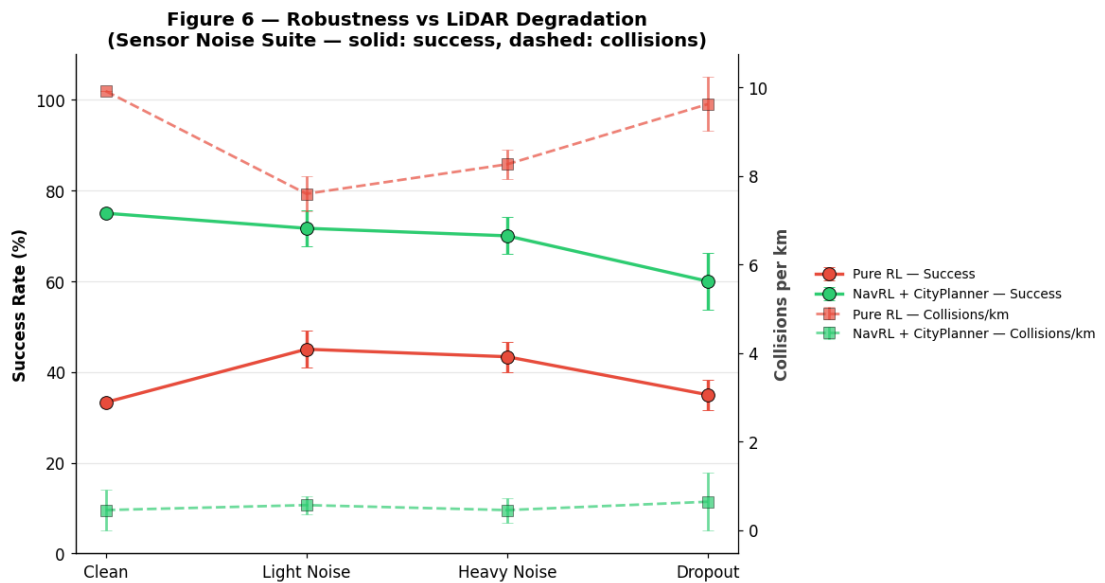


Figure 5.29 — Dual-axis robustness plot: success rate (solid lines, left axis) and collisions/km (dashed lines, right axis) across Clean → Dropout conditions. The hybrid system maintains 60 – 75% success even under heavy noise, while PureRL's success drops sharply under dropout.

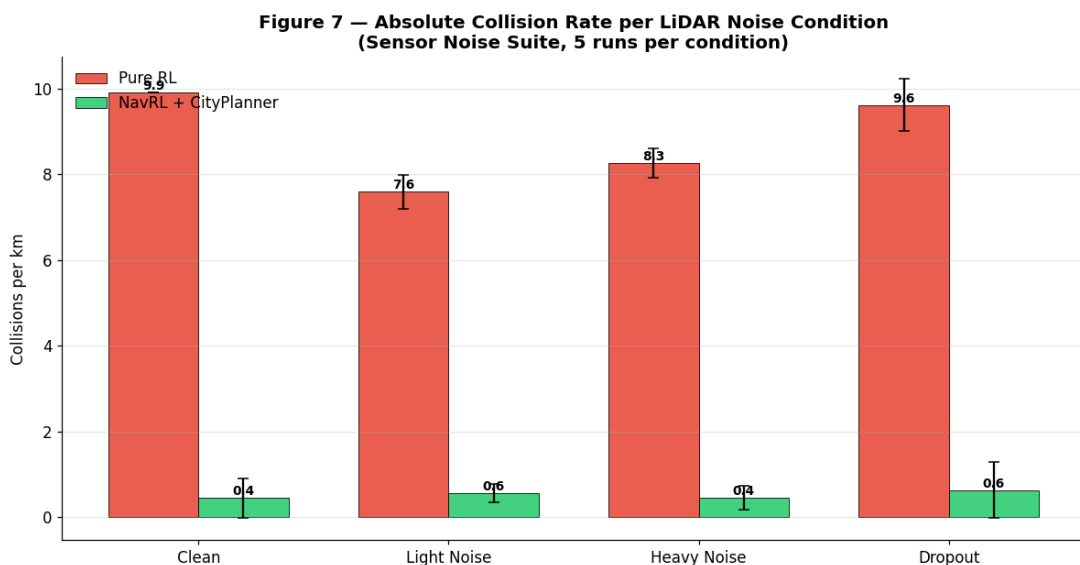


Figure 5.30 — Absolute collision rate per LiDAR noise condition (± 1 std). PureRL collision rates remain 7 – 10 / km across all noise levels, while the hybrid system stays below 1.5 / km through heavy noise.

5.6 Test Coverage Summary

Layer	Tests	Status
Backend services (unit)	6 service test classes; $\geq 70\%$ line coverage	Pass
Backend integration	MockMvc + H2 covering auth / drone / mission / telemetry	Pass
REST contract (Postman)	All 48 endpoints exercised; assertions on JSON schema	Pass
Security	JWT integrity, rate limit, OWASP headers, auth enforcement	Pass
WebSocket telemetry	Latency ≤ 1 s NFR-02 verified	Pass
Frontend widget	Login / Dashboard / List / Detail / Map / Mission	Pass
Frontend integration (E2E)	Full mission flow on emulator	Pass
Autonomy — Standard suite	12-waypoint roam, 5 runs, hybrid 75% / pure 36.66%	Pass
Autonomy — Ablation	5 variants $\times$ 5 runs	Pass
Autonomy — Domain Random.	5 weather conditions	Pass
Autonomy — Sensor Noise	4 LiDAR noise levels $\times$ 5 runs	Pass

Table 5.7 — Combined test coverage summary.

6. Entrepreneurial and Innovation Aspects

6.1 Market and Research Relevance

The global drone-services market is projected to reach USD 63.6 billion by 2030, with urban air mobility, infrastructure inspection, and last-mile delivery as primary growth drivers. The Drone Command Center addresses two simultaneous bottlenecks in this market: (i) the absence of an open, integrated fleet-management dashboard for small-to-medium operators, and (ii) the inability of pure RL navigation systems to operate reliably in dense urban environments. The $2\times$ success and $13\times$ collision-rate improvements demonstrated in §5.5 translate directly to commercial viability metrics: higher delivery completion rates, lower insurance premiums, and reduced regulatory friction.

6.2 Innovation Positioning

The system contributes a novel combination of three open-source ideas:

- **Wrapper-based RL augmentation.** Rather than retraining the NavRL policy (expensive, requiring high-compute infrastructure), the architecture augments a pre-trained checkpoint with a deterministic four-layer planner. This enables urban deployment of existing RL navigation models without additional GPU training cost.
- **Cloud-style operator console for UAVs.** Most open-source UAV stacks expose a single-vehicle Ground Control Station; the Drone Command Center treats fleet-wide telemetry, missions, and commands as first-class REST + WebSocket resources, exposing the same surface to web, mobile, and third-party integrations.
- **Reproducible scientific harness.** The `capstone_test_runner.py` + `analyze_results.py` pipeline produces deterministic CSV summaries and figures from a single command, lowering the barrier for follow-on research to compare planners.

6.3 Ethical and Societal Considerations

- **Safety.** The $13\times$ collision reduction directly mitigates risk to property and pedestrians. The velocity-obstacle shield (Eq. 3.15) provides a hard safety guarantee layer above the neural network.
- **Privacy.** LiDAR-based navigation does not capture identifiable visual information, addressing a key concern in urban UAV deployment regulations.
- **Accessibility.** Building on an open-source research framework [1] enables academic institutions and small operators to deploy state-of-the-art navigation without proprietary infrastructure.
- **Operator accountability.** Every command is logged with the issuing user (Appendix C.8), satisfying audit requirements of common civil aviation regulators.

7. Results and Discussion

7.1 Why Pure RL Fails at the City Scale

The NavRL policy was trained in a 50 m × 50 m arena with sparse random obstacles [1]. The AirSim City environment presents two fundamentally different challenges:

- **Extended obstacles.** Buildings subtend 30–90° of horizontal LiDAR coverage and extend over hundreds of metres. The 4 m reaction horizon is insufficient to route around them.
- **Global route topology.** The optimal path between city waypoints often requires deliberate detours of 50–100 m. Reactive RL always moves toward the goal until blocked.

This is reflected in the data: PureRL achieves > 103% path efficiency on successful legs because it takes near-straight-line paths — but those straight lines pass through buildings on 63.3% of all legs.

7.2 The Hybrid Architecture's Trade-offs

The hybrid system's lower path efficiency (82.5%) and higher time-to-goal (77.94 s vs 20.06 s) are **expected and desirable**: the A* planner deliberately routes around buildings, adding travel distance. The trade-off is clear — longer paths in exchange for successful arrival. The non-zero collision rate of the hybrid system (0.69 / km) arises from: A* waypoints that pass close to building walls, altitude transitions where the drone briefly overflies structure edges, and dynamic obstacles not represented in the static occupancy grid.

7.3 Altitude as a Navigation Dimension

A key insight from the ablation study is that altitude management is **not a secondary concern** but an active navigation strategy. The hybrid planner's altitude range of 2.75 m to 26.86 m (nearly 10× the PureRL range of 2.76 m to 3.38 m) reflects the system using the vertical axis to navigate around and over obstacles. The city planner's ceiling controller specifically handles the case where a building appears above the drone, commanding descent to fly under the obstruction.

7.4 Hardware Design for Real-World Deployment

The NavRL team validated real-time inference on the **NVIDIA Jetson Orin NX** [1], with a total pipeline latency of 65 ms — within the 50 ms control budget at 20 Hz. For the full hybrid stack the additional A* component adds approximately 5 ms, giving a total of ~ 44 ms:

Component	Jetson Orin NX (estimated)
NavRL inference	7 ms
Safety shield	16 ms
Static perception	15 ms
A* planning (2D, 100×100)	~ 5 ms
Pure-Pursuit lookahead	< 1 ms
Total	~ 44 ms

Table 7.1 — Full hybrid-stack runtime estimate on NVIDIA Jetson Orin NX. NavRL module timings from [1]; A* and Pure-Pursuit estimated.

Component	Candidate	Rationale
Compute	Jetson Orin NX 16 GB	CUDA + ROS2, validated by NavRL authors [1]
Frame	Custom CFRP	Generative design — minimum mass
Motors	T-Motor MN5008 KV340	Low vibration, high efficiency
LiDAR	Livox MID-360	360° solid-state, 40 m range, lightweight
Flight Ctrl.	Cube Orange+	MAVLink, ArduPilot, triple IMU
Depth Camera	Intel RealSense D435i	Dynamic obstacle detection (as in [1])
Odometry	FAST-LIO2 [20]	LiDAR-inertial state estimation

Table 7.2 — Indicative hardware BOM for real-world deployment.

7.5 Limitations

- **4 m LiDAR range ceiling.** The NavRL policy cannot react to obstacles beyond 4 m. Adapting the observation to use multi-resolution bins could extend the effective reaction horizon.
- **2D A* planning.** A full 3D voxel map with A* in 3D would better handle variable-height obstacles.
- **No dynamic obstacle awareness in the planner.** The A* path does not account for moving obstacles; the NavRL reactive layer handles these implicitly.
- **Sim-to-real gap for the planner layer.** Real-world LiDAR noise may degrade occupancy-map quality, degrading A* path quality.
- **Single-tenant backend.** The current authentication model is per-instance; multi-tenant isolation would require schema-level partitioning.

8. Conclusion, Future Work and Roadmap

8.1 Summary of Contributions

This capstone deployed and systematically evaluated an end-to-end UAV fleet-management stack consisting of (i) a Spring Boot backend with PostgreSQL, JWT-secured REST + WebSocket APIs, (ii) a Flutter cross-platform operator console, and (iii) an autonomy layer wrapping the NavRL deep reinforcement-learning policy [1] with a deterministic four-layer city planner. The principal results are:

- End-to-end backend with documented REST endpoints, real-time WebSocket telemetry, stateless JWT authentication, Bucket4j rate limiting, and a Hibernate-managed PostgreSQL schema (V1 + V2 migrations).
- Cross-platform Flutter client running on Web, Android, iOS, and Windows from a single codebase.
- Hybrid autonomy achieves **75.00% goal-success rate** versus 36.66% for pure NavRL — a **2× improvement**.
- **Collision rate reduced 13×** (0.69 vs 9.31 per km), the primary safety metric.
- Ablation analysis identifies global path planning as the dominant contributing factor (24× collision-rate reduction when added to RL + AltSM).
- The hybrid system is fully robust to weather perturbations and degrades gracefully under LiDAR noise, maintaining collision rates below 1.5 / km through heavy noise.

8.2 Future Work

- **3D occupancy and planning.** Extend the occupancy grid to a voxel representation, enabling A* to route vertically as well as horizontally.
- **Dynamic obstacle integration into the global plan.** Feed NavRL dynamic obstacle detections into the A* cost map to predict and avoid moving objects.
- **Physical deployment.** Fabricate the platform of Table 7.2, implement the ROS2 integration layer, and validate the hybrid system in an outdoor structured environment.
- **Retraining with extended LiDAR range.** Retrain the NavRL policy with a larger max-ray length (8 – 10 m) to enable earlier, softer avoidance manoeuvres.
- **Multi-tenant SaaS.** Extend the user model to organisations, with row-level security for full multi-tenant isolation.
- **Mobile-first operator app.** Optimise the Flutter UI for tablet field use with offline mission caching.

8.3 Roadmap

Horizon	Theme	Concrete Milestones
Year 1	Hardening and pilot	Hardware integration (Jetson + Pixhawk); IMU noise suite; multi-tenant auth; closed pilot with one operator.

Horizon	Theme	Concrete Milestones
Year 2	Productisation	3D voxel planner; dynamic-obstacle aware planning; live video feed; mobile field app; SOC2-style audit logging; managed cloud deployment (Kubernetes).
Year 3+	Scale-out and certification	Multi-fleet federation; offline / edge inference; certification pre-submission package; partner SDK and marketplace integrations.

Table 8.1 — Roadmap.

References

- [1] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, "NavRL: Learning Safe Flight in Dynamic Environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3668-3675, Apr. 2025. DOI: 10.1109/LRA.2025.3546069.
- [2] S. H. Alsamhi et al., "UAV computing-assisted search and rescue mission framework for disaster and harsh environment mitigation," *Drones*, vol. 6, no. 7, 2022, Art. no. 154.
- [3] Z. Xu, B. Chen, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada, "A vision-based autonomous UAV inspection framework for unknown tunnel construction sites with dynamic obstacles," *IEEE Robot. Automat. Lett.*, vol. 8, no. 8, pp. 4983-4990, Aug. 2023.
- [4] Y. Wang, J. Ji, Q. Wang, C. Xu, and F. Gao, "Autonomous flights in dynamic environments with onboard vision," in *Proc. IEEE/RSJ IROS*, 2021, pp. 1966-1973.
- [5] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, "Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization," in *Proc. IEEE ICRA*, 2023, pp. 1214-1220.
- [6] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478-485, Apr. 2021.
- [7] F. Sadeghi and S. Levine, "CAD2RL: Real single-image flight without a single real image," in *Proc. RSS*, Jul. 2017.
- [8] L. Xie, S. Wang, A. Markham, and N. Trigoni, "Towards monocular vision based obstacle avoidance through deep reinforcement learning," arXiv:1706.09829, 2017.
- [9] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 107-118, Jan. 2021.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [11] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Muller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982-987, 2023.
- [12] T. He, C. Zhang, W. Xiao, G. He, C. Liu, and G. Shi, "Agile but safe: Learning collision-free high-speed legged locomotion," in *Proc. RSS*, Jul. 2024.
- [13] N. Kochdumper et al., "Provably safe reinforcement learning via action projection using reachability analysis and polynomial zonotopes," *IEEE Open J. Control Syst.*, vol. 2,

pp. 79-92, 2023.

[14] Microsoft Research, "AirSim: high-fidelity visual and physical simulation for autonomous vehicles," GitHub repository, 2017–2024.

[15] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," in *Proc. IEEE/RSJ IROS*, 2017, pp. 3948-3955.

[16] P.-W. Chou, D. Maturana, and S. Scherer, "Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution," in *Proc. ICML*, 2017, pp. 834-843.

[17] R. C. Coulter, "Implementation of the pure-pursuit path-tracking algorithm," CMU Robotics Institute Tech. Report CMU-RI-TR-92-01, Jan. 1992.

[18] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760-772, 1998.

[19] Spring Team, "Spring Boot Reference Documentation," VMware, 2024.

[20] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053-2073, Aug. 2022.

[21] Flutter Team, "Flutter Architecture Overview," Google, 2024.

Appendix A — Technology Stack

A.1 Backend Dependencies

Library	Purpose	Version
Spring Boot	Application framework	4.0.2
Spring Data JPA	ORM / database access	Managed by BOM
Spring Security	Authentication, authorization	Managed by BOM
Spring WebSocket	Real-time telemetry	Managed by BOM
Spring Actuator	Health monitoring	Managed by BOM
Spring Mail	Email notifications	Managed by BOM
Spring AMQP	RabbitMQ integration	Managed by BOM
Spring Cache	Caching abstraction	Managed by BOM
PostgreSQL JDBC	Database driver	Managed by BOM
Flyway Core	Schema migration	Managed by BOM
Flyway PostgreSQL	PostgreSQL dialect for Flyway	Managed by BOM
Caffeine	In-memory cache implementation	Managed by BOM
Bucket4j	Token-bucket rate limiting	8.10.1
JWT API	JWT creation and parsing	0.11.5
JWT Impl	JWT implementation	0.11.5
JWT Jackson	JWT Jackson integration	0.11.5
SpringDoc OpenAPI	Swagger UI generation	2.8.4
Lombok	Boilerplate reduction	Managed by BOM
H2 Database	In-memory DB for testing	Test scope

Table A.1 — Backend dependencies.

A.2 Frontend Dependencies

Library	Purpose	Version
flutter_riverpod	State management	^2.4.9
riverpod_annotation	Code-generation annotations	^2.3.3
dio	HTTP client with interceptors	^5.4.0
web_socket_channel	WebSocket client	^2.4.0
shared_preferences	Local preferences storage	^2.2.2
flutter_secure_storage	Encrypted JWT storage	^9.0.0
fl_chart	Telemetry line / bar charts	^0.66.0
flutter_map	Interactive map (OpenStreetMap)	^6.1.0
latlong2	Geographic coordinate types	^0.9.0
lottie	JSON-based animations	^3.0.0
shimmer	Loading skeleton effects	^3.0.0
animate_do	Widget entrance animations	^3.1.2
intl	Date / number formatting	^0.18.1
go_router	Declarative navigation	^13.0.1
equatable	Value equality for models	^2.0.5
json_annotation	JSON serialisation annotations	^4.8.1
google_fonts	Rajdhani + Space Mono fonts	^6.1.0
iconsax	Tactical icon set	^0.0.8
flutter_svg	SVG asset rendering	^2.0.9

Table A.2 — Frontend dependencies.

Appendix B — REST API Endpoint Catalogue

B.1 Authentication (/api/auth)

Method	Path	Auth	Role	Description
POST	/api/auth/register	No	—	Register a new user account
POST	/api/auth/login	No	—	Authenticate; receive JWT + refresh token
POST	/api/auth/refresh	No	—	Exchange a refresh token for a new JWT (rotates the refresh token)
POST	/api/auth/logout	Yes	Authenticated	Revoke the current refresh token
GET	/api/auth/me	Yes	Authenticated	Get the current authenticated user profile
GET	/api/auth/validate	Yes	Authenticated	Validate the bearer JWT (returns 200 / true if valid)

The legacy email-based password-reset endpoints (/forgot-password, /reset-password) were removed in the V2 demo cleanup. Credential recovery is now performed out-of-band by the system administrator.

B.2 Drone Management (/api/drones)

Method	Path	Auth	Role	Description
POST	/api/drones	Yes	Authenticated	Register a new drone
GET	/api/drones	Yes	Authenticated	List all drones (paginated)
GET	/api/drones/all	Yes	Authenticated	List all drones (unpaginated)
GET	/api/drones/{id}	Yes	Authenticated	Get drone by UUID
PUT	/api/drones/{id}	Yes	Authenticated	Update drone details
DELETE	/api/drones/{id}	Yes	Authenticated	Delete drone
GET	/api/drones/status/{status}	Yes	Authenticated	Filter by connection status
GET	/api/drones/flight-status/{status}	Yes	Authenticated	Filter by flight status

B.3 Mission Management (/api/missions)

Meth od	Path	Aut h	Role	Description
POST	/api/missions	Yes	Authenticated	Create a new mission
GET	/api/missions/{id}	Yes	Authenticated	Get mission by UUID
GET	/api/missions	Yes	Authenticated	List all missions (paginated)
GET	/api/missions/status/{s tatus}	Yes	Authenticated	Filter missions by status
GET	/api/missions/drone/{dr oneId}	Yes	Authenticated	List missions for a drone
PUT	/api/missions/{id}	Yes	Authenticated	Update mission
DELE TE	/api/missions/{id}	Yes	Authenticated	Delete mission
POST	/api/missions/{id}/star t	Yes	Authenticated	Start mission
POST	/api/missions/{id}/paus e	Yes	Authenticated	Pause mission
POST	/api/missions/{id}/resu me	Yes	Authenticated	Resume mission
POST	/api/missions/{id}/comp lete	Yes	Authenticated	Mark mission complete
POST	/api/missions/{id}/abor t	Yes	Authenticated	Abort mission
GET	/api/missions/{id}/wayp oints	Yes	Authenticated	Get mission waypoints
POST	/api/missions/{id}/wayp oints	Yes	Authenticated	Add waypoint to mission
DELE TE	/api/missions/{id}/wayp oints/{wId}	Yes	Authenticated	Remove waypoint

B.4 Commands (/api/commands)

Meth od	Path	Aut h	Role	Description
POST	/api/commands	Yes	Authenticated	Issue a command to a drone
GET	/api/commands/drone/{dr oneId}	Yes	Authenticated	Get command history for drone
PUT	/api/commands/{id}/stat us	Yes	Authenticated	Update command status

B.5 Telemetry (/api/telemetry)

Meth od	Path	Aut h	Role	Description
POST	/api/telemetry	No	—	Ingest telemetry data from drone

Method	Path	Auth	Role	Description
GET	/api/telemetry/drone/{id}/latest	Yes	Authenticated	Get latest telemetry record
GET	/api/telemetry/drone/{id}	Yes	Authenticated	Get telemetry history (paginated)
GET	/api/telemetry/drone/{id}/range	Yes	Authenticated	Get telemetry in time range
GET	/api/telemetry/drone/{id}/flight-path	Yes	Authenticated	Get flight-path coordinates

B.6 User Management (/api/users)

Meth od	Path	Aut h	Role	Description
GET	/api/users	Yes	Authenticated	List all users
GET	/api/users/{id}	Yes	Authenticated	Get user by UUID
PUT	/api/users/{id}	Yes	Authenticated	Update user
DELE TE	/api/users/{id}	Yes	Authenticated	Deactivate user

B.7 System (/actuator)

Meth od	Path	Aut h	Role	Description
GET	/actuator/health	No	—	Application health status (publicly exposed for liveness probes)
GET	/actuator/info	No	—	Application build information
GET	/actuator/metrics	Yes	Authenticated	JVM and application metrics
GET	/actuator/loggers	Yes	Authenticated	Logger level inspection / management
GET	/actuator/caches	Yes	Authenticated	Caffeine cache inspection

B.8 AirSim Bridge / NavRL / Logs

All endpoints under `/api/airsim/**`, `/api/navrl/**` and `/api/logs/**` require an authenticated JWT. They expose the AirSim co-process bridge controls, NavRL evaluation triggers, and operational log retrieval used by the dashboard's diagnostics view. Their full schemas are available at `/swagger-ui/index.html`.

B.9 WebSocket

Endpoint	Protocol	Description
<code>/ws/telemetry</code>	WebSocket / SockJS	Real-time telemetry broadcast. All subscribers receive telemetry JSON when data is ingested.

Table B.1 — WebSocket endpoint.

Appendix C — Database Schema Reference

C.1 Table: *users*

Column	Type / Constraints	Description
id	UUID — PK, DEFAULT gen_random_uuid()	User unique identifier
username	VARCHAR(50) NOT NULL UNIQUE	Login username
password	VARCHAR(255) NOT NULL	BCrypt hashed password
email	VARCHAR(100) NOT NULL UNIQUE	Email address
enabled	BOOLEAN NOT NULL DEFAULT true	Account active flag
created_at	TIMESTAMPTZ DEFAULT NOW()	Registration timestamp

C.2 Table: *refresh_tokens*

Column	Type / Constraints	Description
id	UUID PK	Refresh-token row identifier
token	VARCHAR(255) NOT NULL UNIQUE	Opaque refresh-token string returned to the client
user_id	UUID FK → users(id) ON DELETE CASCADE	Owning user
expiry_date	TIMESTAMPTZ NOT NULL	Expiry timestamp (default: 7 days after issuance)
revoked	BOOLEAN NOT NULL DEFAULT false	Set true on logout / rotation
created_at	TIMESTAMPTZ DEFAULT NOW()	Issuance timestamp

The original V1 schema also defined *user_roles*, *user_drone_assignments* and *password_reset_tokens* tables. All three were removed by migration *V2__demo_cleanup.sql*; the live deployment uses a flat authenticated-user model and out-of-band credential recovery.

C.3 Table: *drones*

Column	Type / Constraints	Description
id	UUID PK	Drone unique identifier
serial_number	VARCHAR(50) NOT NULL UNIQUE	Hardware serial number
name	VARCHAR(100) NOT NULL	Display name
model_type	VARCHAR(100) NOT NULL	Manufacturer model string
firmware_version	VARCHAR(50)	Current firmware version
connection_status	VARCHAR(50) NOT NULL	CONNECTED / DISCONNECTED / etc.
flight_status	VARCHAR(50) NOT NULL	IDLE / IN_FLIGHT / etc.
battery_level	DOUBLE DEFAULT 100.0	Battery %
latitude	DOUBLE DEFAULT 0.0	Current latitude
longitude	DOUBLE DEFAULT 0.0	Current longitude
altitude	DOUBLE DEFAULT 0.0	Current altitude (metres)
autonomy_level	VARCHAR(50)	MANUAL / SEMI_AUTONOMOUS / AUTONOMOUS
navigation_mode	VARCHAR(50)	GPS / WAYPOINT / MANUAL
failsafe_enabled	BOOLEAN DEFAULT true	Failsafe system active
obstacle_detected	BOOLEAN DEFAULT false	Obstacle avoidance flag
last_heartbeat	TIMESTAMPZ	Last heartbeat received
registered_at	TIMESTAMPZ DEFAULT NOW()	Registration time
home_latitude	DOUBLE	RTH home latitude
home_longitude	DOUBLE	RTH home longitude
home_altitude	DOUBLE	RTH home altitude

C.4 Table: missions

Column	Type / Constraints	Description
id	UUID PK	Mission identifier
name	VARCHAR(100) NOT NULL	Mission name
description	TEXT	Mission description
status	VARCHAR(50) NOT NULL	Mission status enum
priority	INTEGER DEFAULT 0	Priority (higher = more urgent)
start_time	TIMESTAMPTZ	Actual start time
end_time	TIMESTAMPTZ	Actual end time
estimated_duration_minutes	INTEGER	Planned duration
actual_duration_minutes	INTEGER	Actual duration
created_at	TIMESTAMPTZ DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	Last update timestamp
assigned_drone_id	UUID FK → drones(id)	Assigned drone
created_by_id	UUID FK → users(id)	Creating user

C.5 Table: waypoints

Column	Type / Constraints	Description
id	UUID PK	Waypoint identifier
latitude	DOUBLE NOT NULL	Waypoint latitude
longitude	DOUBLE NOT NULL	Waypoint longitude
altitude	DOUBLE NOT NULL	Waypoint altitude (metres)
sequence_order	INTEGER NOT NULL	Order in mission sequence
action	VARCHAR(50)	HOVER / TAKE_PHOTO / SCAN
hover_duration_seconds	INTEGER DEFAULT 0	Time to hover at waypoint
speed	DOUBLE	Target speed (m/s)
heading	DOUBLE	Target heading (degrees)

Column	Type / Constraints	Description
reached	BOOLEAN DEFAULT false	Reached confirmation
reached_at	TIMESTAMPTZ	Time waypoint was reached
mission_id	UUID FK → missions(id) ON DELETE CASCADE	Parent mission

C.6 Table: telemetry

Column	Type / Constraints	Description
id	UUID PK	Record identifier
timestamp	TIMESTAMPTZ	Measurement time
latitude	DOUBLE	Position latitude
longitude	DOUBLE	Position longitude
altitude	DOUBLE	Altitude (metres)
speed	DOUBLE	Ground speed (m/s)
heading	DOUBLE	Heading (degrees 0–360)
battery_level	DOUBLE	Battery %
signal_strength	DOUBLE	Signal strength %
gps_satellites	INTEGER	Satellites in view
temperature	DOUBLE	Ambient temperature (°C)
humidity	DOUBLE	Relative humidity %
wind_speed	DOUBLE	Wind speed (m/s)
wind_direction	DOUBLE	Wind direction (degrees)
flight_mode	VARCHAR(50)	Active flight mode
drone_id	UUID FK → drones(id)	Source drone

C.7 Table: sensors

Column	Type / Constraints	Description
id	UUID PK	Sensor identifier
name	VARCHAR(100)	Sensor name
type	VARCHAR(50)	CAMERA / LIDAR / GPS / IMU / etc.
status	VARCHAR(50)	ACTIVE / INACTIVE / ERROR / CALIBRATING
last_reading	TEXT	Last sensor reading (JSON)
last_reading_at	TIMESTAMPTZ	Time of last reading
drone_id	UUID FK → drones(id)	Parent drone

C.8 Table: commands

Column	Type / Constraints	Description
id	UUID PK	Command identifier
command_type	VARCHAR(50)	TAKEOFF / LAND / RTH / etc.
status	VARCHAR(50)	PENDING / SENT / EXECUTED / FAILED
payload	TEXT	JSON command parameters

Column	Type / Constraints	Description
response	TEXT	Drone response message
issued_at	TIMESTAMPTZ	When command was issued
executed_at	TIMESTAMPTZ	When command was sent to drone
completed_at	TIMESTAMPTZ	When drone confirmed execution
drone_id	UUID FK → drones(id)	Target drone
issued_by_id	UUID FK → users(id)	Issuing user

Appendix D — Enumeration Reference

Enum	Values
ConnectionStatus	CONNECTED, DISCONNECTED, CONNECTING, ERROR, UNKNOWN
FlightStatus	IDLE, TAKING_OFF, IN_FLIGHT, LANDING, HOVERING, RETURNING_HOME, EMERGENCY, OFFLINE
MissionStatus	PLANNED, IN_PROGRESS, COMPLETED, ABORTED, FAILED, PAUSED
CommandType	TAKEOFF, LAND, RETURN_TO_HOME, HOVER, GO_TO_WAYPOINT, START_MISSION, ABORT_MISSION, EMERGENCY_STOP, SET_ALTITUDE, SET_SPEED, ROTATE, TAKE_PHOTO, START_STREAMING, STOP_STREAMING
CommandStatus	PENDING, SENT, ACKNOWLEDGED, EXECUTED, FAILED, CANCELLED
WaypointAction	HOVER, TAKE_PHOTO, SCAN, INSPECT, DELIVER, LAND, NONE
SensorType	CAMERA, LIDAR, GPS, BAROMETER, IMU, RADAR, THERMAL_CAMERA, ULTRASONIC, MAGNETOMETER
AutonomyLevel	MANUAL, SEMI_AUTONOMOUS, AUTONOMOUS, EMERGENCY
NavigationMode	MANUAL, GPS, WAYPOINT, FOLLOW_ME, ORBIT, RETURN_HOME

Table D.1 — System enumerations.

Appendix E — Frontend Screen Inventory

Screen	Route	Description
Splash Screen	/	Animated boot screen with JWT validation redirect
Login Screen	/login	Authentication form with validation
Register Screen	/register	New account creation form
Dashboard	/dashboard	Fleet overview: stat cards, drone-list summary
Drone List	/drones	Paginated drone list with status indicators
Drone Detail	/drones/:id	Telemetry charts, sensor grid, command panel
Mission List	/missions	Mission list with status badges
Mission Detail	/missions/:id	Mission overview, waypoint list, status controls
Create Mission	/missions/create	Mission creation form with waypoint builder
Map Screen	/map	Interactive OpenStreetMap with drone markers
Settings	/settings	User preferences and logout

Table E.1 — Flutter screen inventory.

Appendix F — Configuration Reference

F.1 Key application.properties Parameters

Key	Default / Example	Description
server.port	8080	HTTP listening port
spring.datasource.url	jdbc:postgresql://localhost:5432/drone_db	Database URL
spring.jpa.hibernate.ddl-auto	update	Live schema authority — Hibernate auto-applies entity changes
spring.flyway.enabled	false	Disabled at runtime; V1 + V2 were applied manually during setup
spring.flyway.locations	classpath:db/migration	Migration script location (V1__Initial_schema.sql, V2__demo_cleanup.sql)
jwt.secret	(env JWT_SECRET)	JWT signing secret (min 256-bit)
jwt.expiration	3600000	JWT expiry in ms (1 h)
app.jwt.refresh-token-expiration-days	7	Refresh-token validity (days)
spring.cache.type	caffeine	Cache provider
app.email.enabled	false	Outbound mail disabled in this build
app.rabbitmq.enabled	false	RabbitMQ disabled; AmqpAutoConfiguration excluded
app.airsim.bridge.auto-start	true	Auto-launch the Python AirSim bridge process on startup
management.endpoints.web.exposure.include	health,info,metrics,loggers,caches	Exposed Actuator endpoints

Table F.1 — Backend configuration keys.

F.2 Environment Variables (Secrets)

The following values must **never** be committed to source control and must be supplied via environment variables or a secrets manager. The mail and RabbitMQ variables are only consumed when the corresponding app.email.enabled / app.rabbitmq.enabled feature flags are turned on (both default to false in this build).

- JWT_SECRET — JWT HMAC signing key (minimum 32 characters; required).
- DB_USERNAME / DB_PASSWORD — PostgreSQL credentials.
- CORS_ALLOWED_ORIGINS — comma-separated allow-list of front-end origins.
- AIRSIM_PYTHON_PATH / AIRSIM_SCRIPT_PATH — absolute paths to the Python interpreter and the AirSim bridge script auto-launched at startup.
- RABBITMQ_HOST / RABBITMQ_PORT / RABBITMQ_USERNAME / RABBITMQ_PASSWORD — only required if app.rabbitmq.enabled=true.

Appendix G — Document Control

Version	Date	Author	Description
0.1	2026-04-01	Development Team	Initial draft (software engineering view).
0.5	2026-04-15	Development Team	Architecture and requirements complete.
0.9	2026-04-22	Development Team	AI evaluation suite integrated.
1.0	2026-04-26	Development Team	Final combined version for submission.

Table G.1 — Document revision history.

End of report. Base RL framework: NavRL, Zhefan Xu et al. [1]. Hybrid city planner, test harness, evaluation, backend, and frontend implementation: Capstone Team. Simulation environment: Microsoft AirSim [14].